CovertTux: A Cache Policy-oblivious Timing Covert Channel on a Linux System

Dhruv Gupta - BT/CSE/220361

Pragati Agrawal - BT/CSE/220779

Supervisors:

Prof. Mainak Chaudhuri

Prof. Debadatta Mishra

Ms. Yashika Verma (Mentor)

Department of Computer Science and Engineering
Indian Institute of Technology Kanpur

April 18, 2025

Introduction

Introduction

- What is a timing based covert channel?

Introduction

- What is a timing based covert channel?
- A sender and a receiver agree on a predefined protocol and exploit the latency difference between a LLC hit and a LLC miss to communicate covertly.
- The LLC is typically shared across various cores of a chip-multiprocessor. So they are scheduled on two different cores but use the shared LLC to establish the covert channel.

Overview

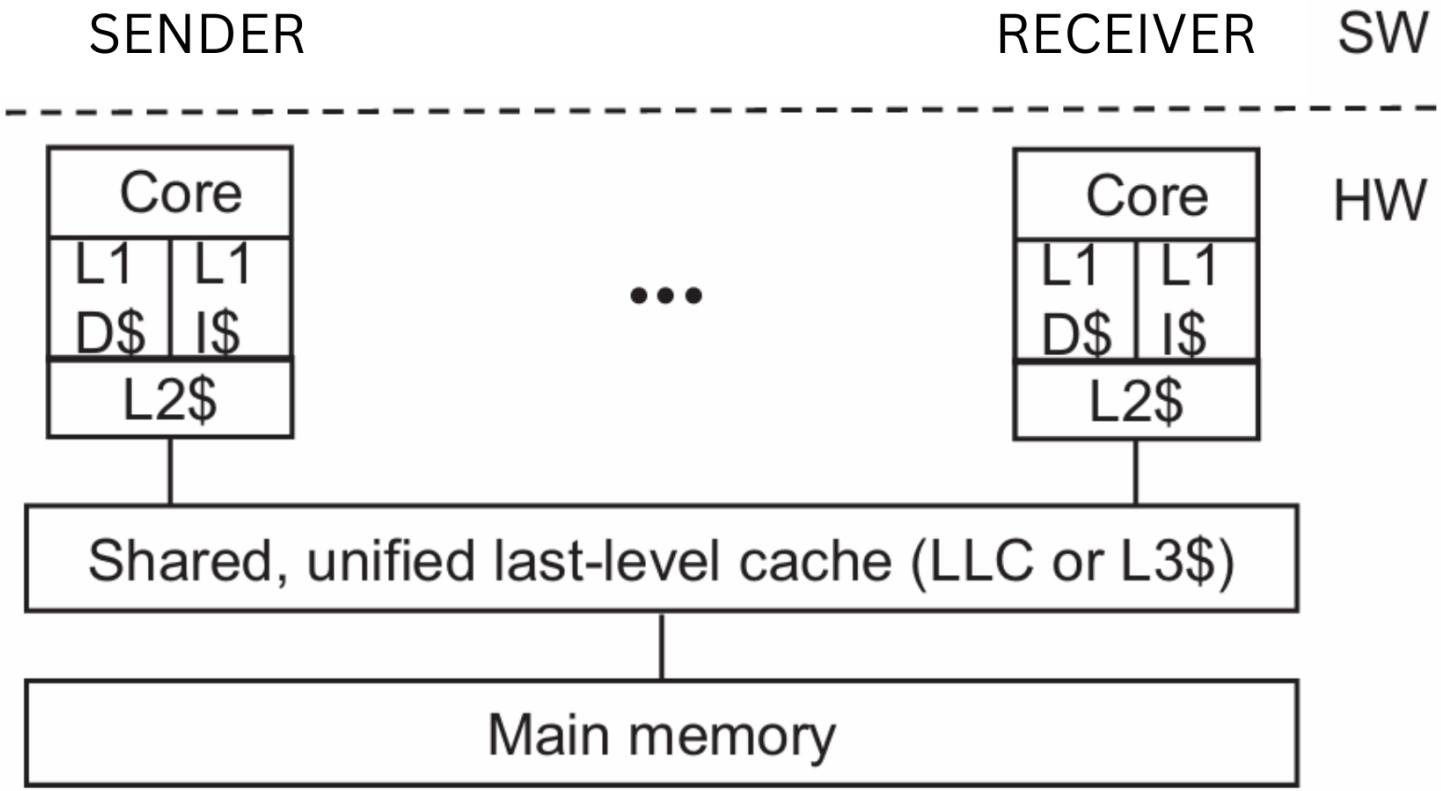


Figure: Covert channel in a real system[1]

[1] Source : https://ieeexplore.ieee.org/document/7163050

Overview

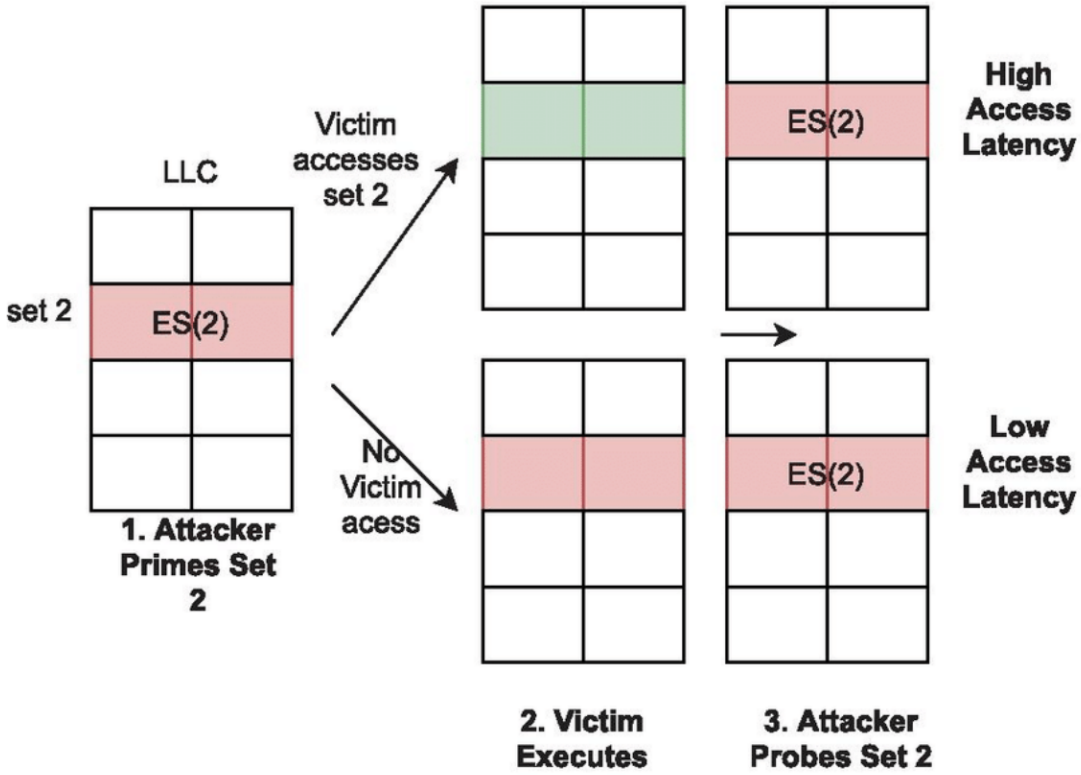


Figure: Prime+Probe Attack [2]

[2] Source : https://www.researchgate.net/publication/351710688_A_Survey_on_Cache_Timing_Channel_Attacks_for_Multicore_Processors

Scope of our UGP

Scope of our UGP

- **LeakyRand** provides a covert channel for fully associative cache with random replacement policy which is efficient and has a very low bit error rate.

Scope of our UGP

- **LeakyRand** provides a covert channel for fully associative cache with random replacement policy which is efficient and has a very low bit error rate.
- **CovertCraft** (our last UGP) demonstrated its simpler version on a Spartan 3E FPGA board.

Scope of our UGP

- **LeakyRand** provides a covert channel for fully associative cache with random replacement policy which is efficient and has a very low bit error rate.
- **CovertCraft** (our last UGP) demonstrated its simpler version on a Spartan 3E FPGA board.
- **CovertTux** utilizes the idea used by LeakyRand to establish a cache policy oblivious covert channel in a real Linux system.

Scope of our UGP

- **LeakyRand** provides a covert channel for fully associative cache with random replacement policy which is efficient and has a very low bit error rate.
- **CovertCraft** (our last UGP) demonstrated its simpler version on a Spartan 3E FPGA board.
- **CovertTux** utilizes the idea used by LeakyRand to establish a cache policy oblivious covert channel in a real Linux system.
- Any replacement policy would work.

Scope of our UGP

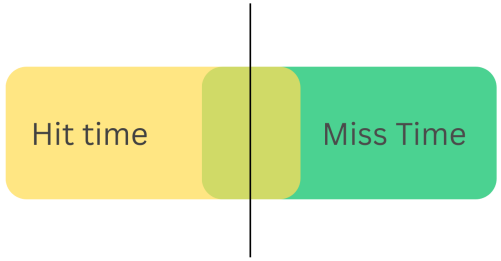
- **LeakyRand** provides a covert channel for fully associative cache with random replacement policy which is efficient and has a very low bit error rate.
- **CovertCraft** (our last UGP) demonstrated its simpler version on a Spartan 3E FPGA board.
- **CovertTux** utilizes the idea used by LeakyRand to establish a cache policy oblivious covert channel in a real Linux system.
- Any replacement policy would work.
- A set of a set-associative LLC works.

Our Machine Configurations

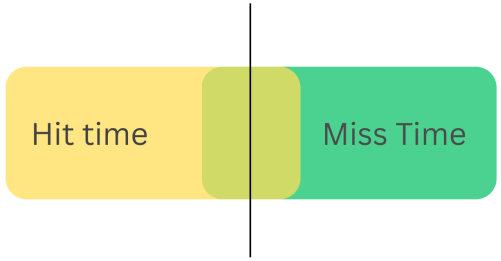
Our Machine Configurations

- We used one of our CSE server machines - 172.27.21.1
- It is a Intel(R) Xeon(R) E-2278G CPU running at 3.40GHz having 16 logical cores
- It has a three level cache hierarchy.
- L1i Cache : 32 KB, 8-ways
- L1d Cache : 32 KB, 8-ways
- L2 Cache : 256 KB, 4-ways, non-inclusive
- L3 Cache (LLC) : 16 MB, 16-ways, **inclusive**
- L1 dTLB reach: 256 KB
- L2 TLB reach: 6MB

Challenges: Finding Threshold

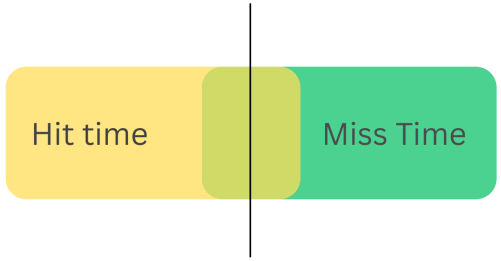


Challenges: Finding Threshold



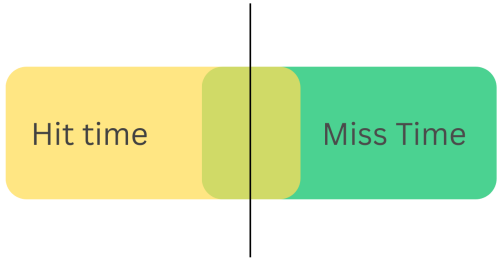
- Non-Deterministic hit/miss latencies.

Challenges: Finding Threshold



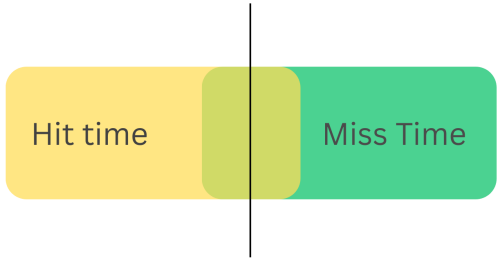
- Non-Deterministic hit/miss latencies.
- There can be an overlap between hit and miss latencies, i.e., Max Hit Latency $\geq$ Min Miss Latency.

Challenges: Finding Threshold



- Non-Deterministic hit/miss latencies.
- There can be an overlap between hit and miss latencies, i.e., Max Hit Latency $\geq$ Min Miss Latency.
- Measuring hit latency for LLC. Need to ensure block misses in L1 and L2 cache but not suffer a TLB miss.

Challenges: Finding Threshold



- Non-Deterministic hit/miss latencies.
- There can be an overlap between hit and miss latencies, i.e., Max Hit Latency $\geq$ Min Miss Latency.
- Measuring hit latency for LLC. Need to ensure block misses in L1 and L2 cache but not suffer a TLB miss.
- Context switches create outliers in data.

Finding Threshold of Hit v/s Miss

Finding Threshold of Hit v/s Miss

- We accessed an array of size 1MB, 4096 times (total 64M accesses) and collected the hit and miss times for each block.

Finding Threshold of Hit v/s Miss

- We accessed an array of size 1MB, 4096 times (total 64M accesses) and collected the hit and miss times for each block.
- **Why 1MB?**

Finding Threshold of Hit v/s Miss

- We accessed an array of size 1MB, 4096 times (total 64M accesses) and collected the hit and miss times for each block.
- **Why 1MB?**
- For **hit time**, first access the entire array and then time access each block.

Finding Threshold of Hit v/s Miss

- We accessed an array of size 1MB, 4096 times (total 64M accesses) and collected the hit and miss times for each block.
- **Why 1MB?**
- For **hit time**, first access the entire array and then time access each block.
- For **miss time**, flush a block, and then time access it.

Finding Threshold of Hit v/s Miss

- We accessed an array of size 1MB, 4096 times (total 64M accesses) and collected the hit and miss times for each block.
- **Why 1MB?**
- For **hit time**, first access the entire array and then time access each block.
- For **miss time**, flush a block, and then time access it.
- Collected this data 5-6 times and found a suiteable threshold which would give minimum errors.

Finding Threshold of Hit v/s Miss

- We accessed an array of size 1MB, 4096 times (total 64M accesses) and collected the hit and miss times for each block.
- **Why 1MB?**
- For **hit time**, first access the entire array and then time access each block.
- For **miss time**, flush a block, and then time access it.
- Collected this data 5-6 times and found a suiteable threshold which would give minimum errors.
- Typical LLC Hit time = 100 rdtsc cycles, Typical Miss time = 300+ rdtsc cycles

Finding Threshold of Hit v/s Miss

- We accessed an array of size 1MB, 4096 times (total 64M accesses) and collected the hit and miss times for each block.
- **Why 1MB?**
- For **hit time**, first access the entire array and then time access each block.
- For **miss time**, flush a block, and then time access it.
- Collected this data 5-6 times and found a suiteable threshold which would give minimum errors.
- Typical LLC Hit time = 100 rdtsc cycles, Typical Miss time = 300+ rdtsc cycles
- Our **Threshold = 200** rdtsc cycles
  Hit correctly classified = 99.9964%
  Miss correctly classified = 99.9999%

Challenges: Finding Threshold

- Prefetcher can identify the access pattern and cause false hits.

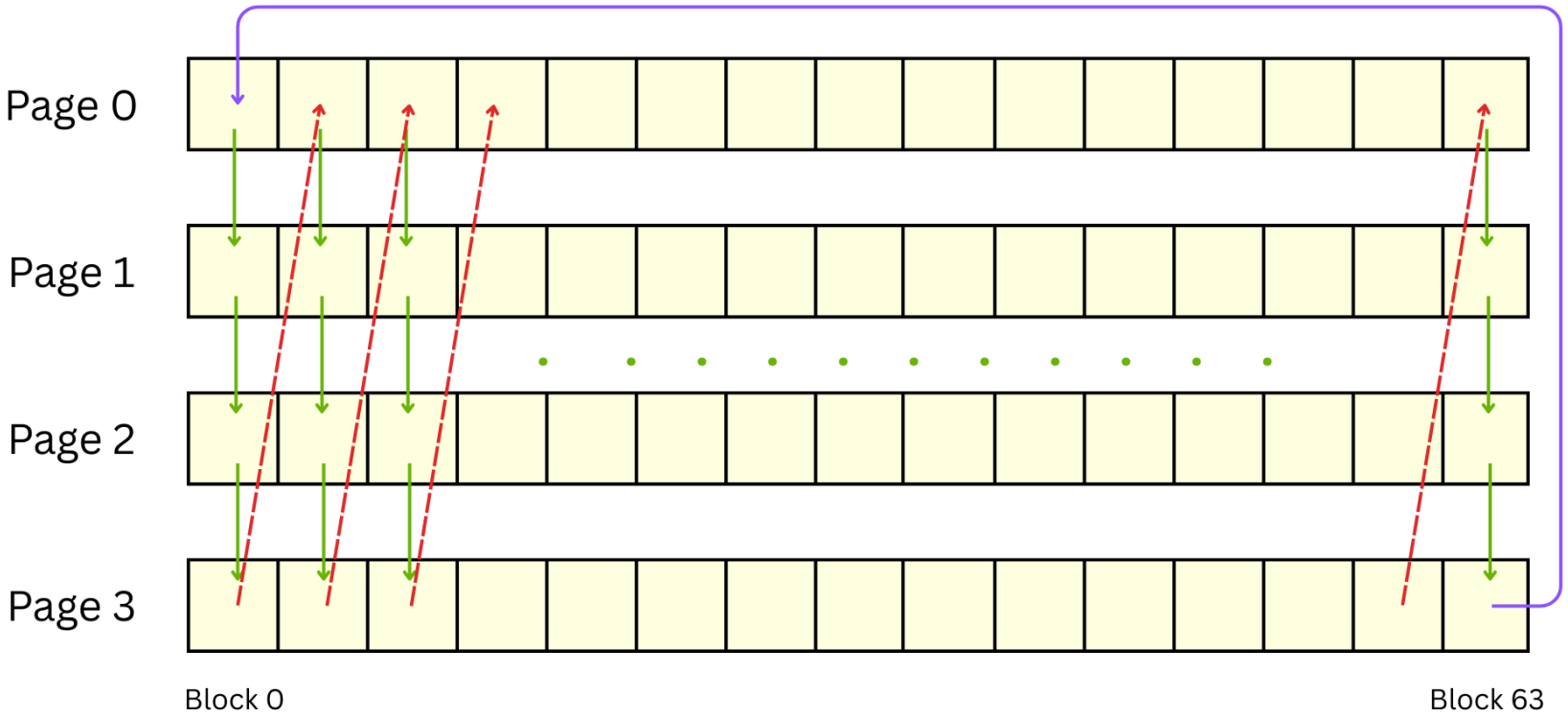


Figure: Access Pattern to fool the prefetcher

Challenges: Finding Eviction Set and Disturbance Set

Challenges: Finding Eviction Set and Disturbance Set

- Unknown Hashing function and V2P address translations.

Challenges: Finding Eviction Set and Disturbance Set

- Unknown Hashing function and V2P address translations.
- Need to isolate a relatively quiet set

Challenges: Finding Eviction Set and Disturbance Set

- Unknown Hashing function and V2P address translations.
- Need to isolate a relatively quiet set
- **Noise** may cause false misses.

Challenges: Finding Eviction Set and Disturbance Set

- Unknown Hashing function and V2P address translations.
- Need to isolate a relatively quiet set
- **Noise** may cause false misses.
- **Prefetcher** can identify access pattern and prefetch our blocks and cause them to hit when we expect a miss.

Challenges: Finding Eviction Set and Disturbance Set

- Unknown Hashing function and V2P address translations.
- Need to isolate a relatively quiet set
- **Noise** may cause false misses.
- **Prefetcher** can identify access pattern and prefetch our blocks and cause them to hit when we expect a miss.
- Sender needs to find disturbance set size elements mapping to the same cache set which receiver isolated, from a different process.

Challenges: Synchronising sender and receiver

[3]Source : https://www.tutorialspoint.com/inter_process_communication/inter_process_communication_pipes.htm

Challenges: Synchronising sender and receiver

- Sender and receiver need to know when they can do their work, and exactly one should work at any point in time for all steps.

[3] Source : https://www.tutorialspoint.com/inter_process_communication/inter_process_communication_pipes.htm

Challenges: Synchronising sender and receiver

- Sender and receiver need to know when they can do their work, and exactly one should work at any point in time for all steps.
- We have used pipes between sender and receiver to synchronize them.

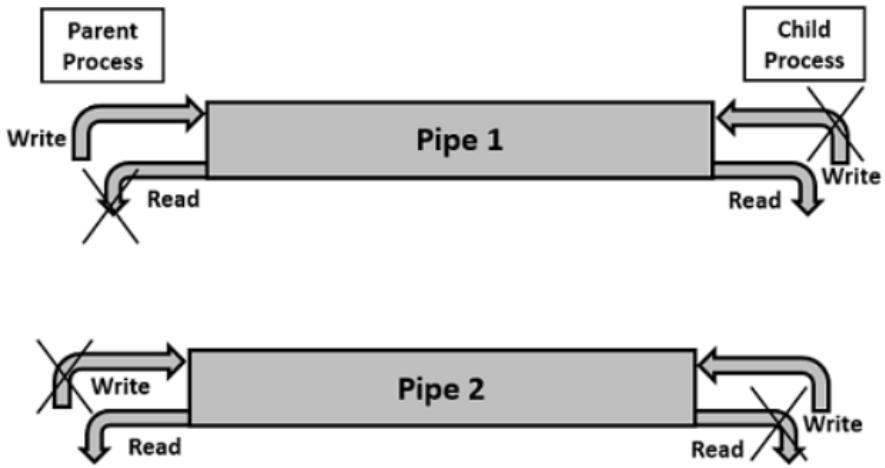


Figure: Inter Process Communication Pipe [3]

[3] Source : https://www.tutorialspoint.com/inter_process_communication/inter_process_communication_pipes.htm

Overview of Steps

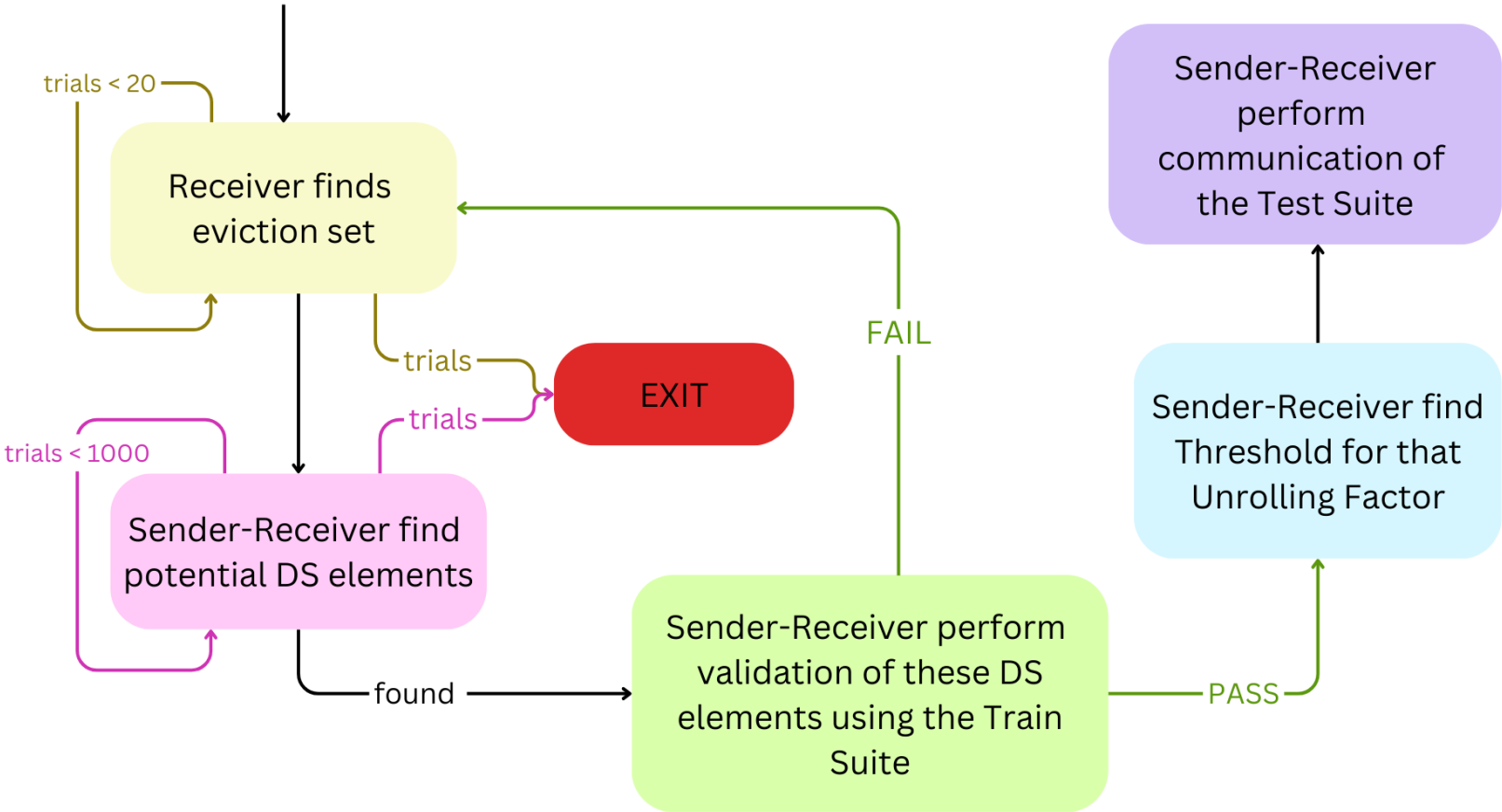


Figure: Steps for establishing Covert Channel

Finding an eviction set for the receiver

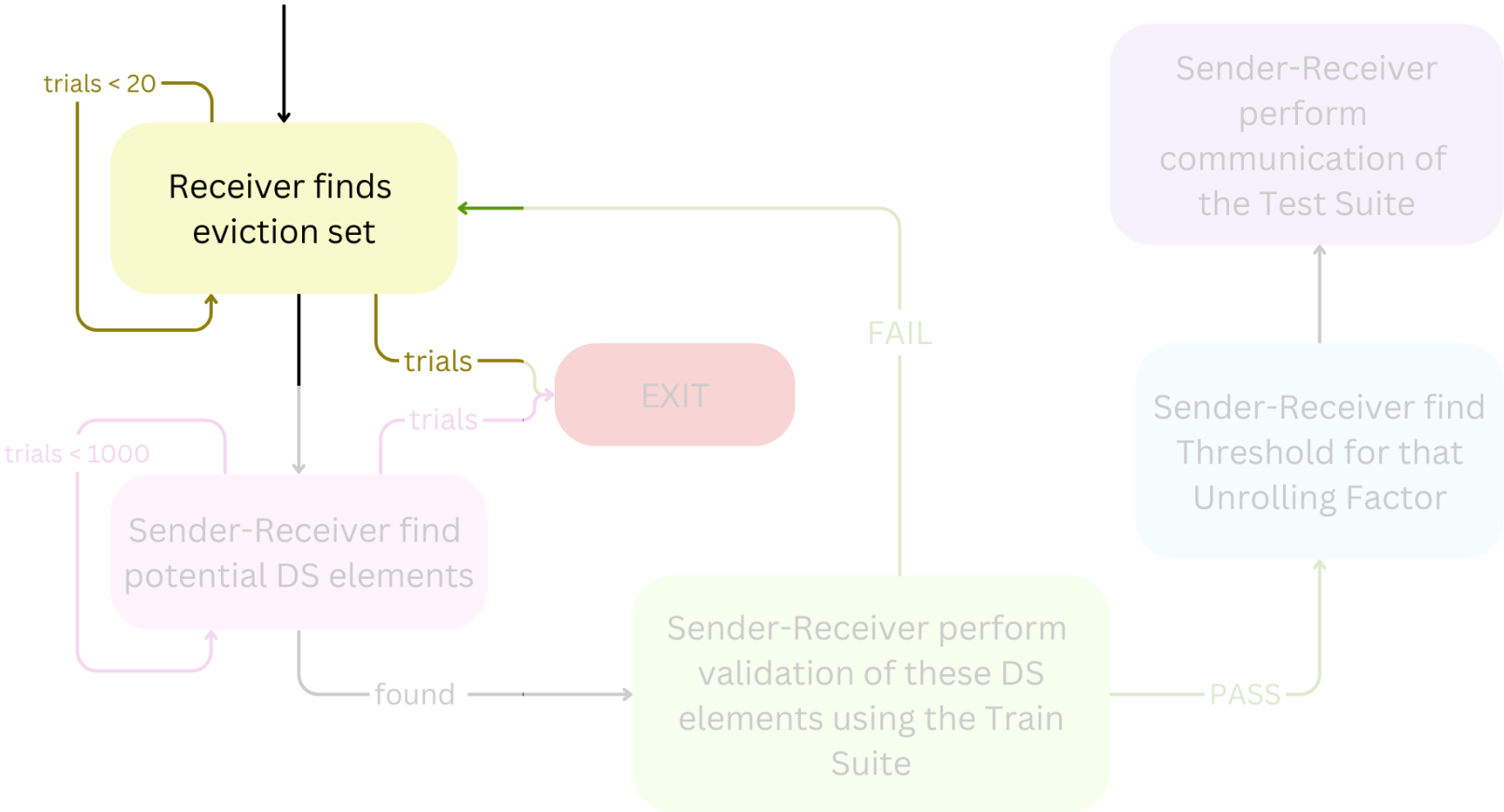


Figure: EV set for receiver

Finding an eviction set for the receiver

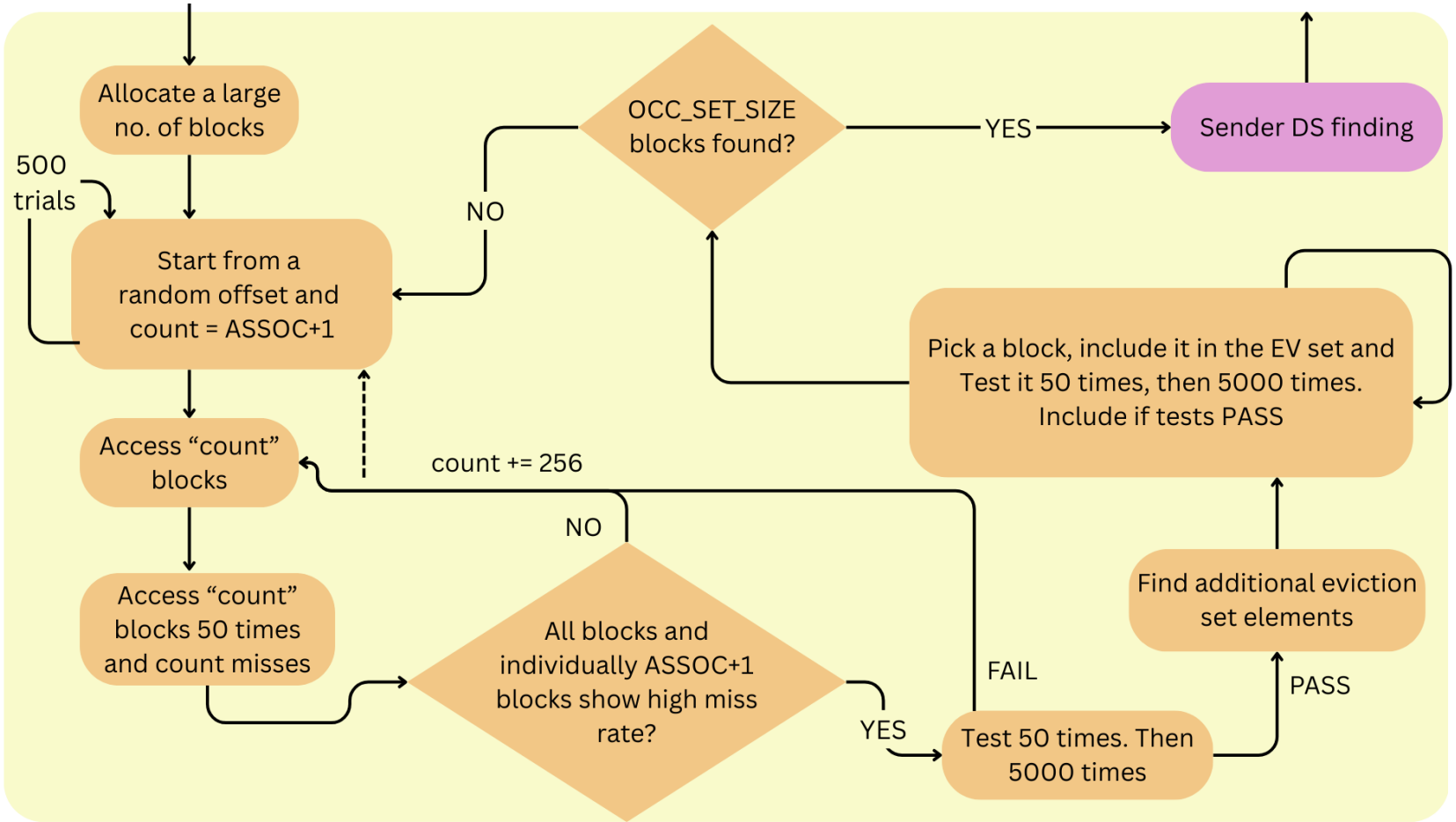


Figure: EV set for receiver

Finding disturbance set for the sender

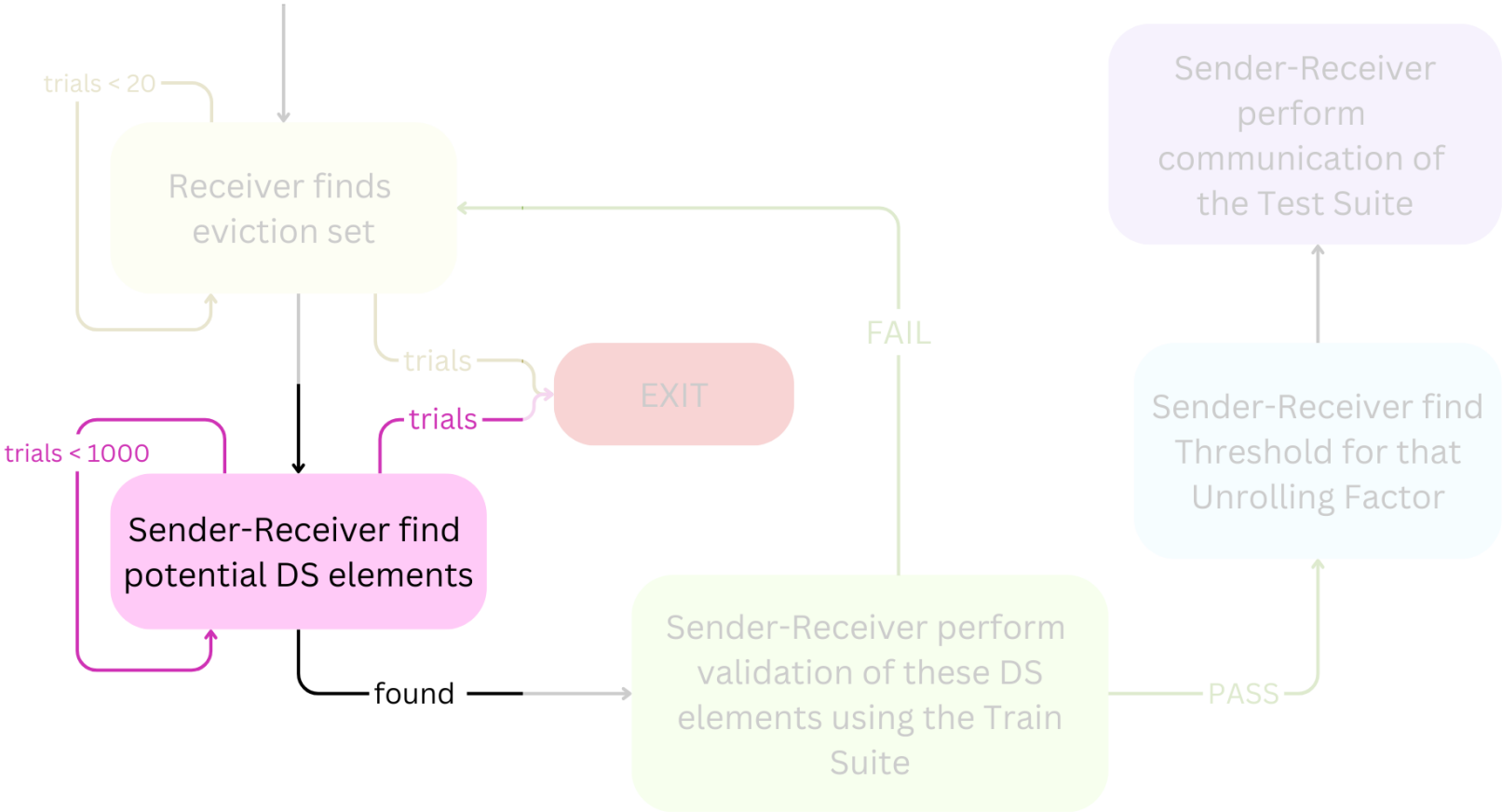


Figure: DS for sender

Finding disturbance set for the sender

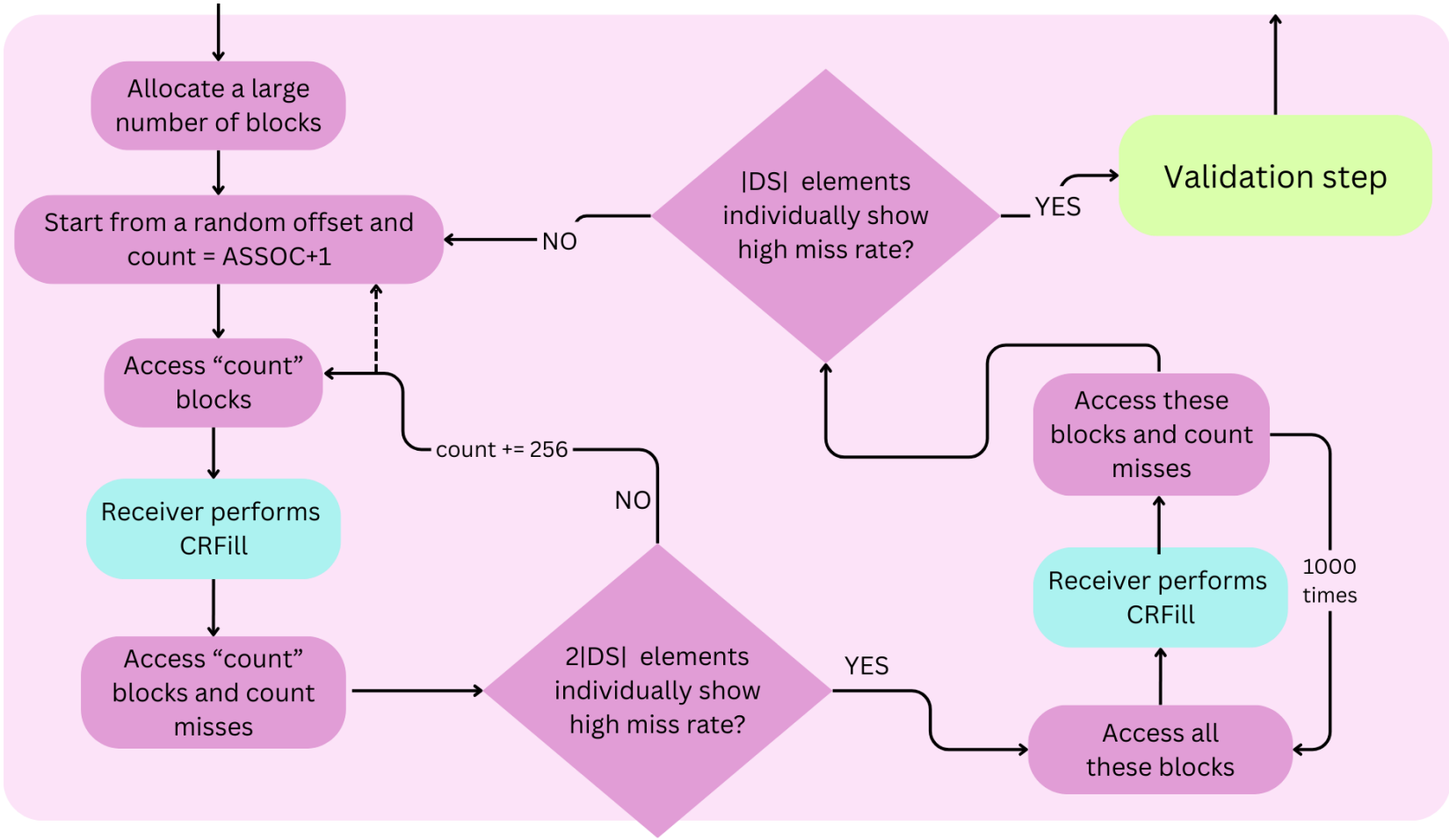


Figure: DS for sender

Validation of disturbance set using Train_suite

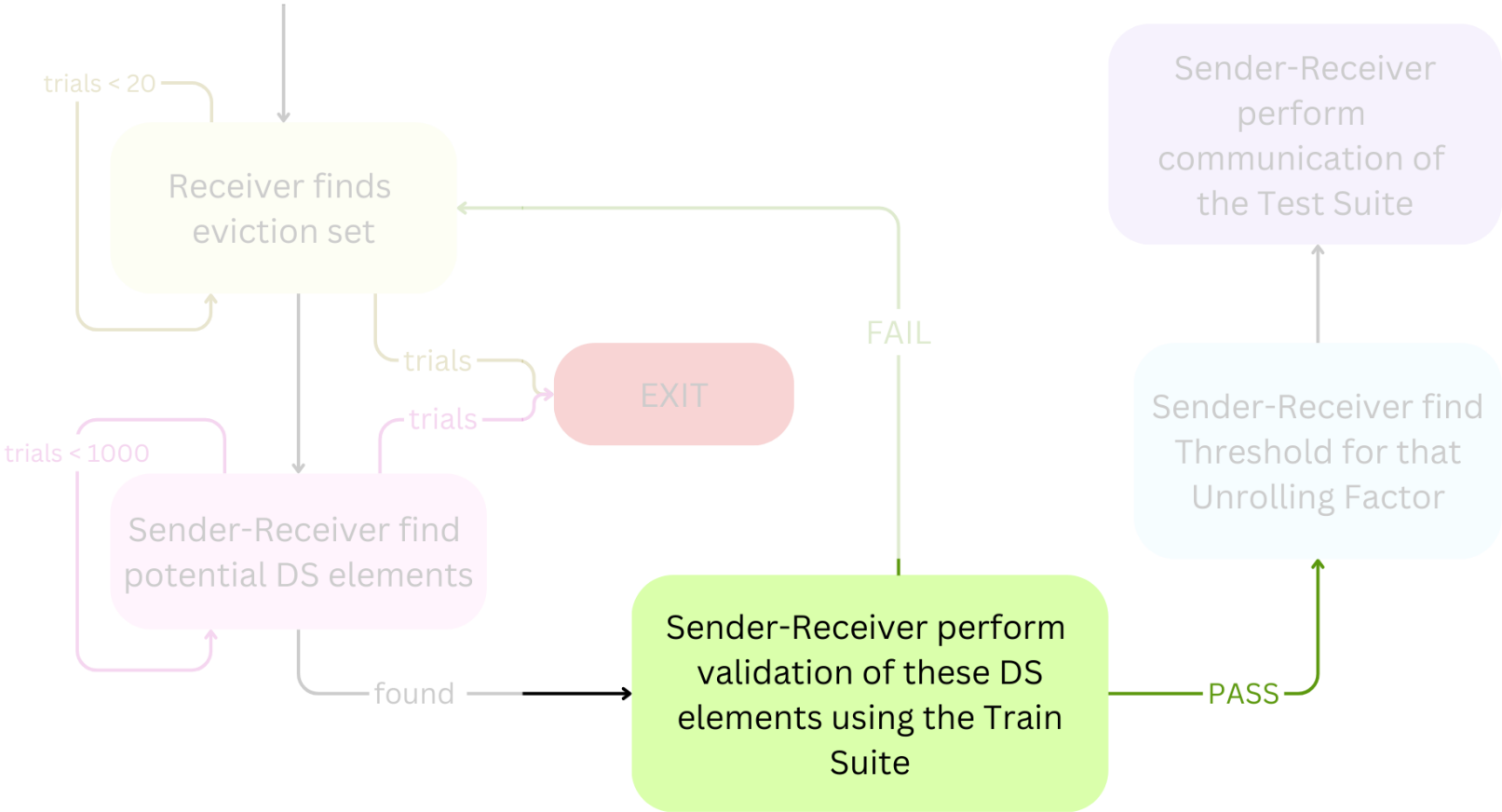


Figure: Validation step

Validation of disturbance set using Train_suite

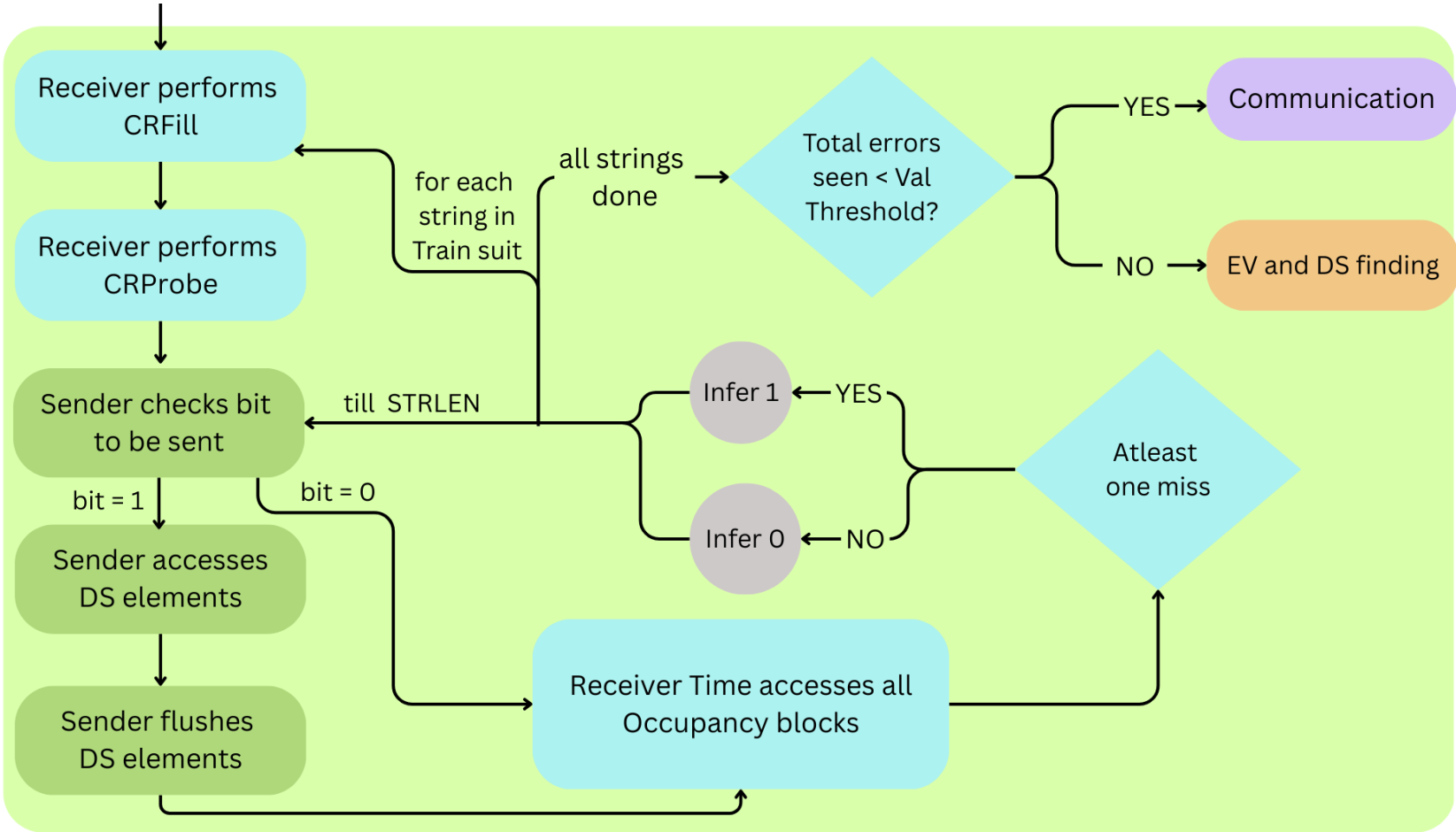


Figure: Validation step

Finding Unrolling Factor Threshold for receiver

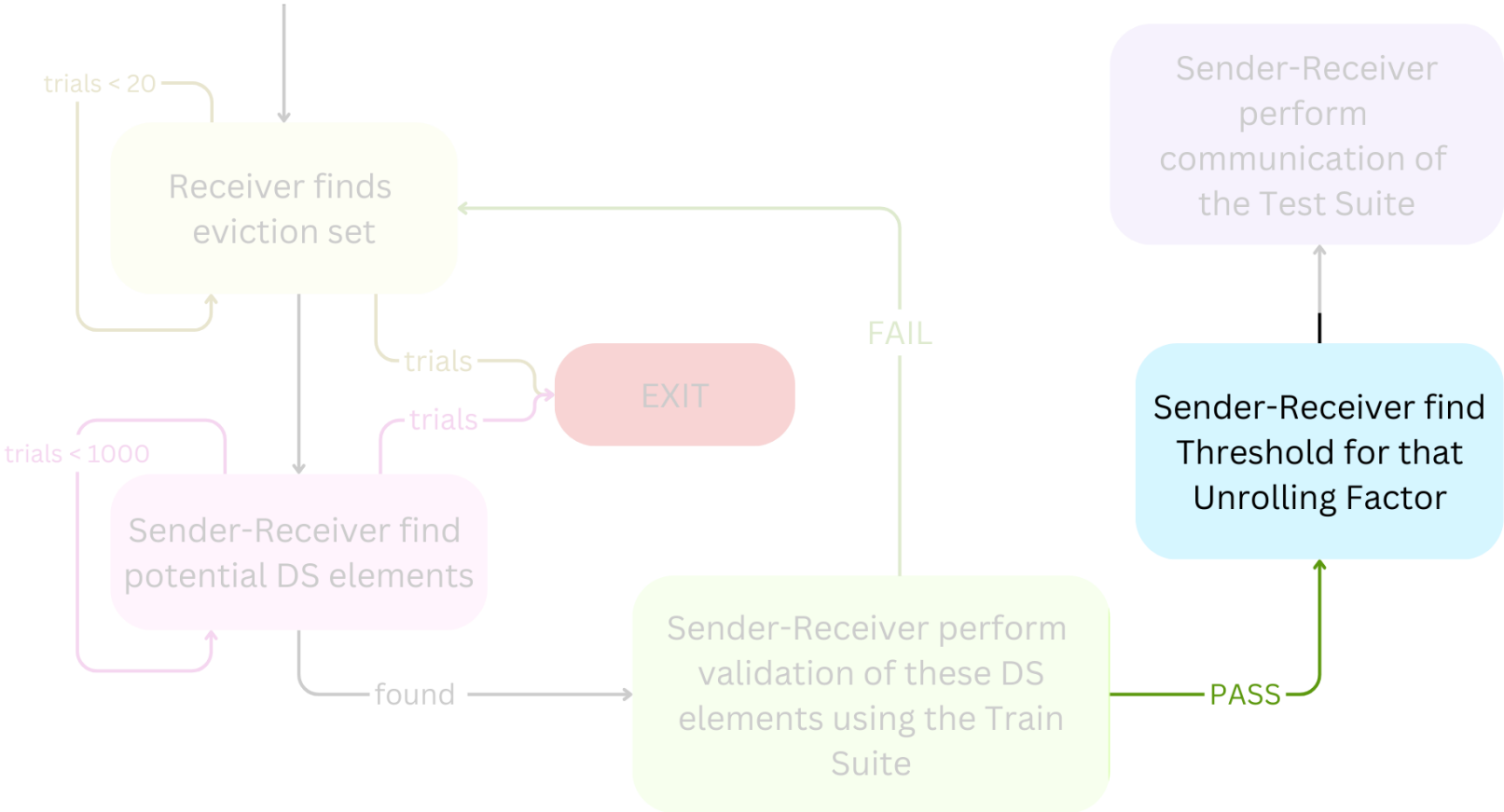


Figure: UF Threshold

Finding Unrolling Factor Threshold for receiver

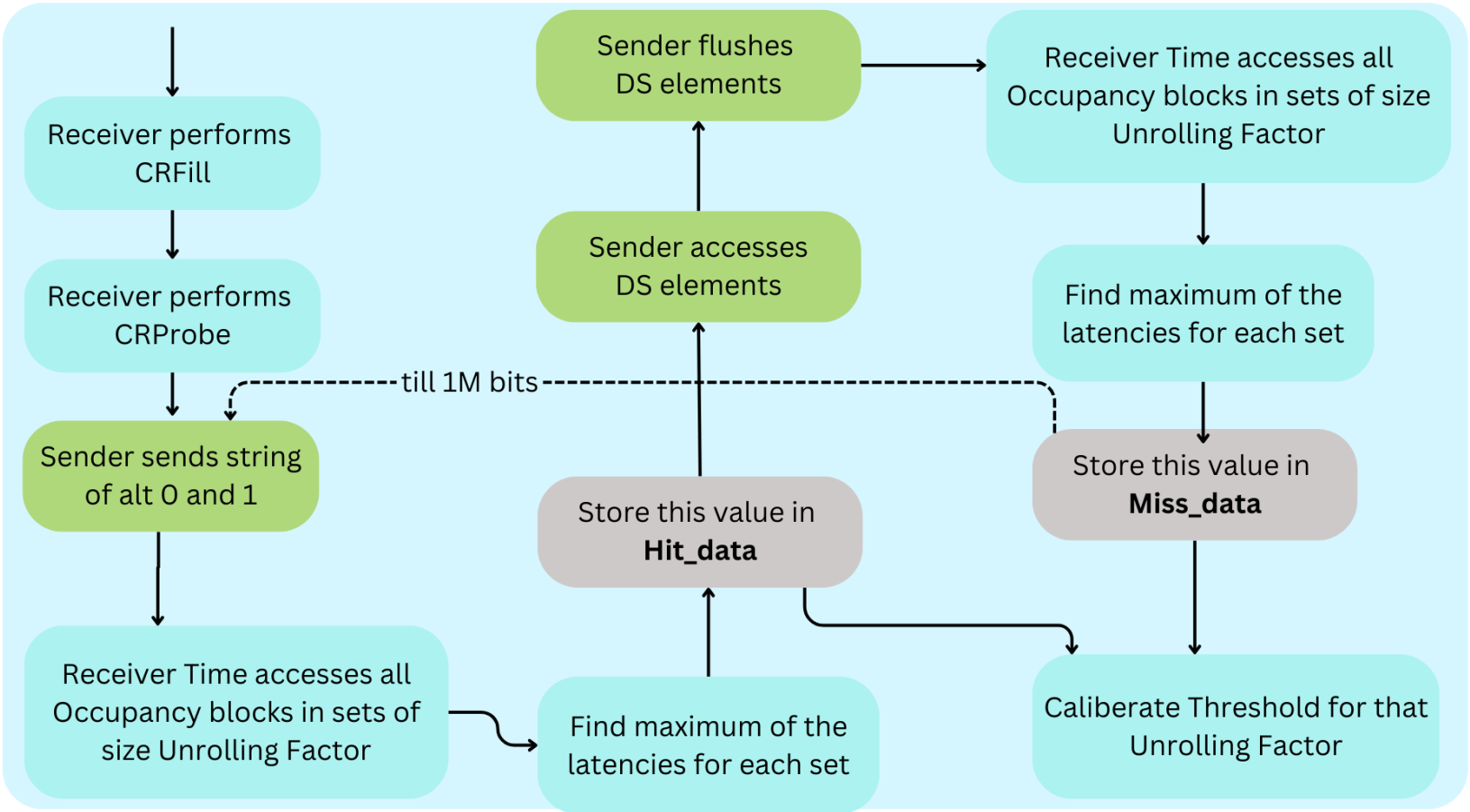


Figure: UF Threshold

Final Communication of Test_suite

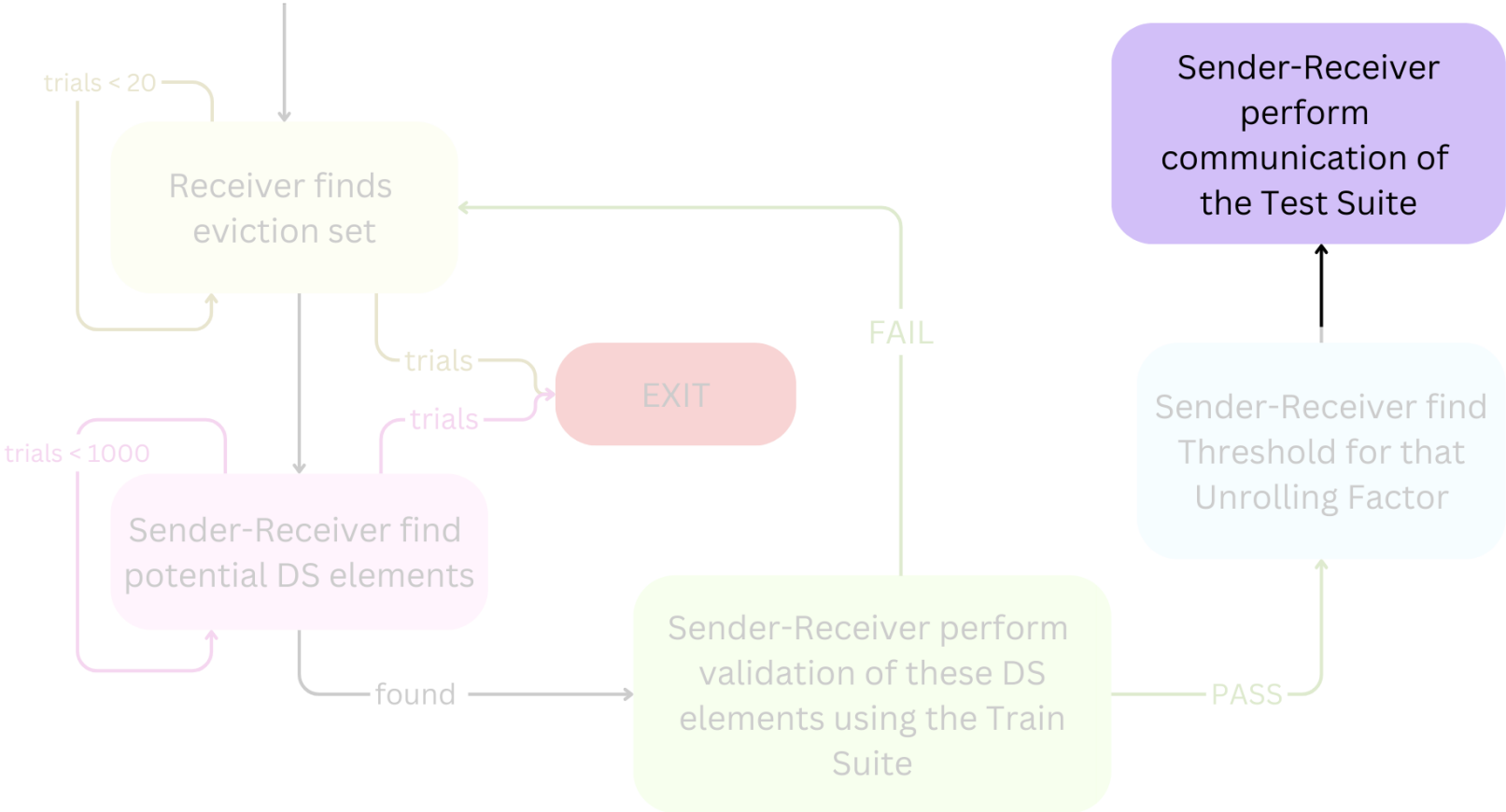


Figure: Communication

Final Communication of Test_suite

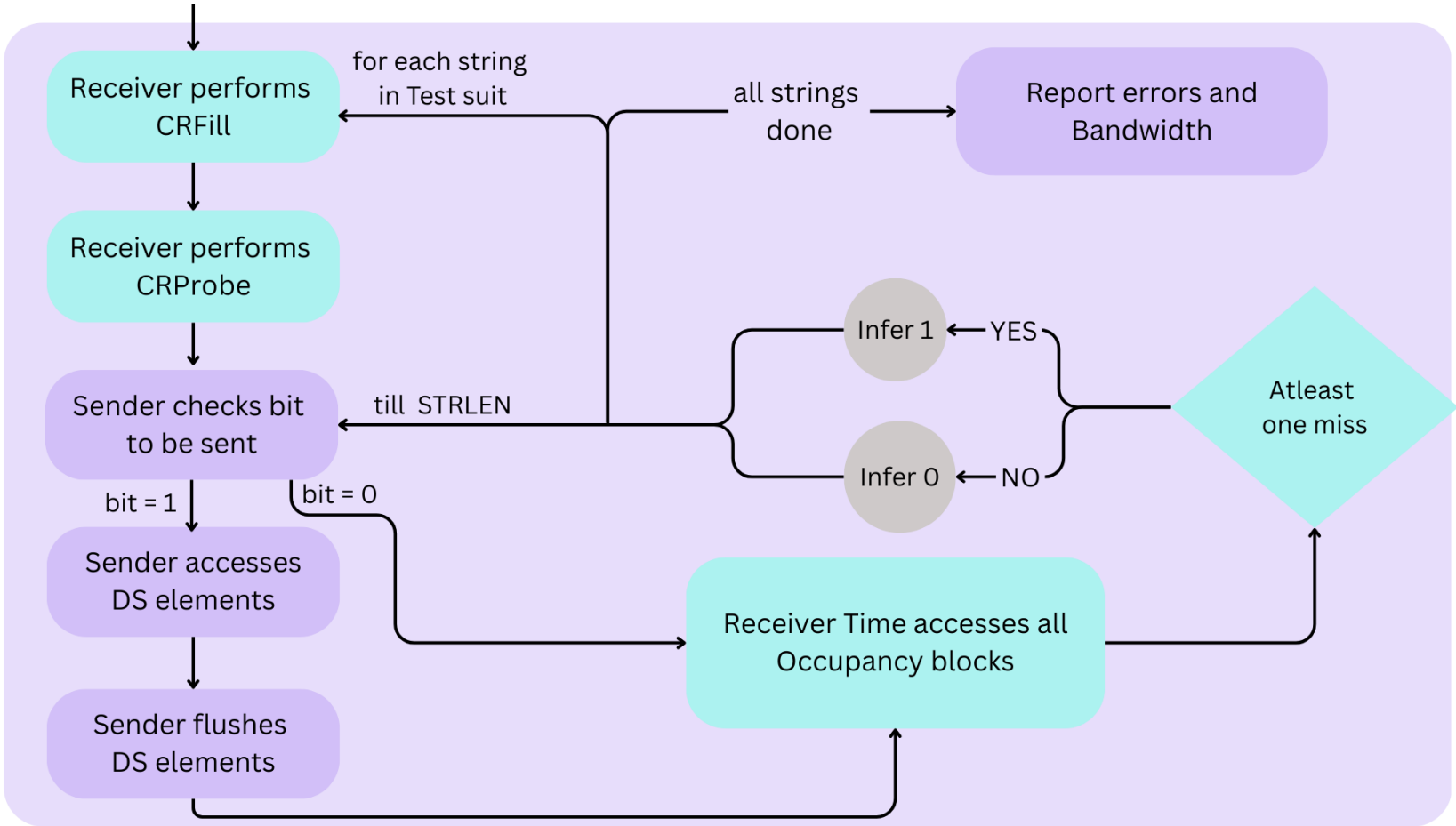


Figure: Communication

Cause of Errors (0 → 1)

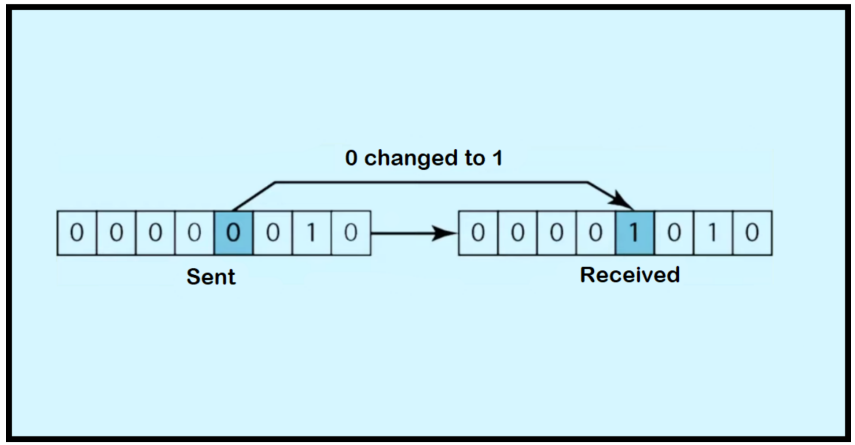


Figure: 0 to 1 errors

Cause of Errors (0 → 1)

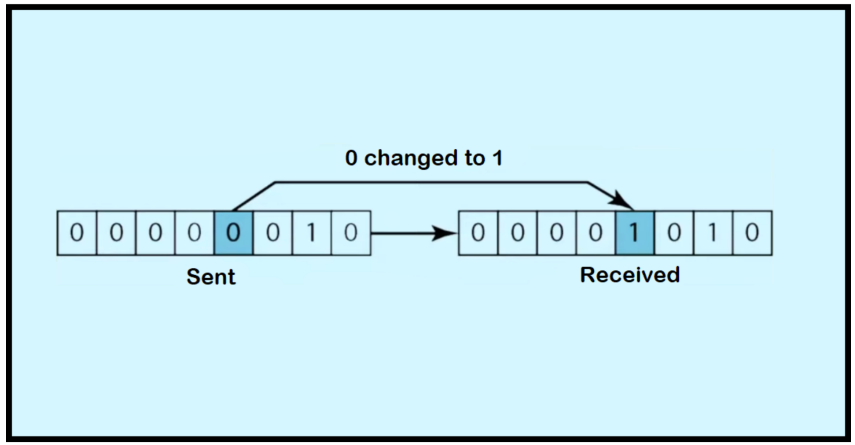


Figure: 0 to 1 errors

- Can occur due to noise during communication

Cause of Errors (0 → 1)

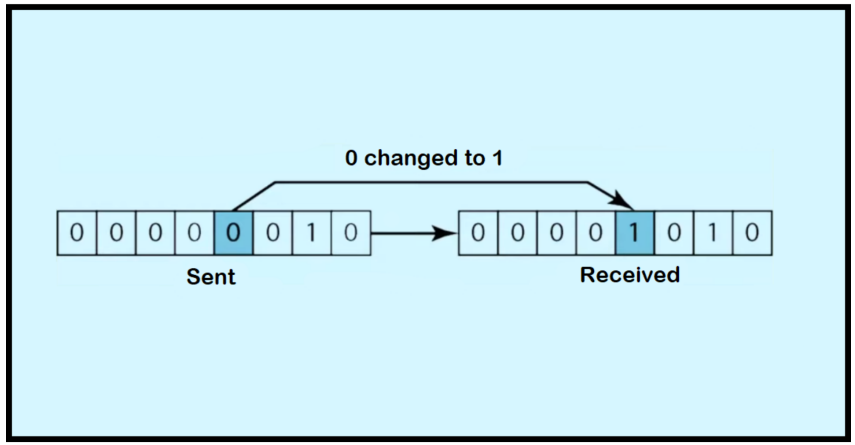


Figure: 0 to 1 errors

- Can occur due to noise during communication
- OS noise - context switches, syscalls, page faults

Cause of Errors (0 → 1)

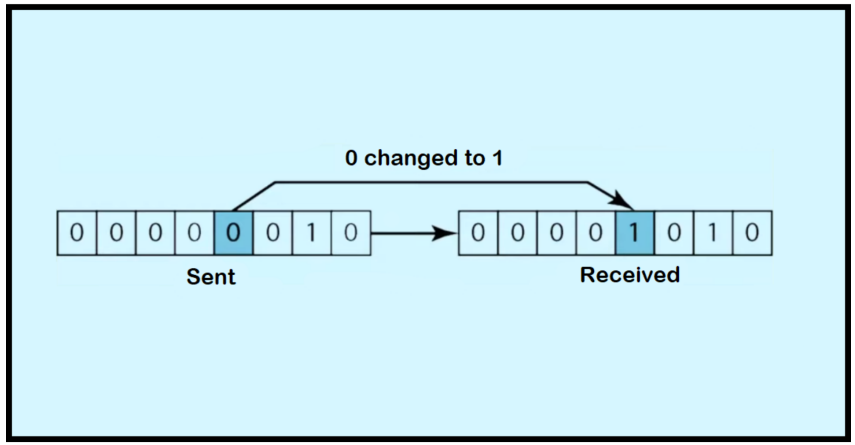


Figure: 0 to 1 errors

- Can occur due to noise during communication
- OS noise - context switches, syscalls, page faults
- Other processes accessing memory (if any)

Cause of Errors (0 → 1)

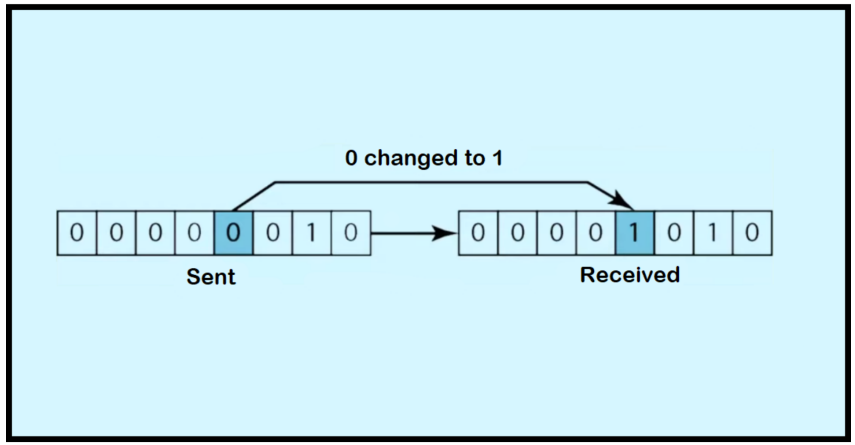


Figure: 0 to 1 errors

- Can occur due to noise during communication
- OS noise - context switches, syscalls, page faults
- Other processes accessing memory (if any)
- Our own data blocks and code blocks

Cause of Errors (0 → 1)

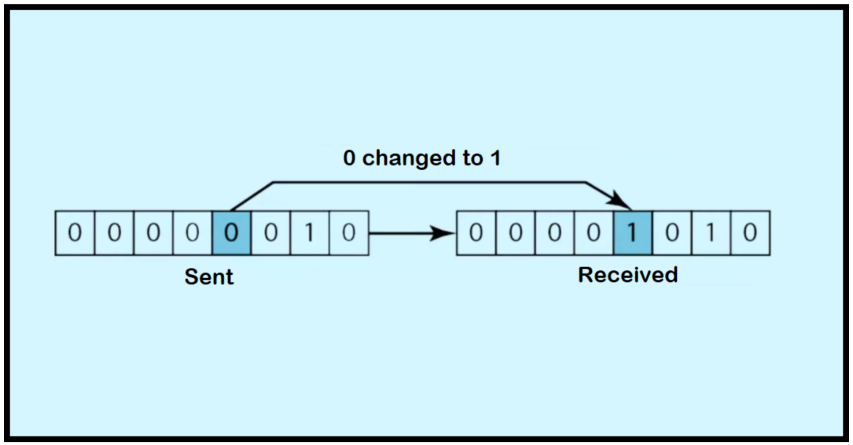


Figure: 0 to 1 errors

- Can occur due to noise during communication
- OS noise - context switches, syscalls, page faults
- Other processes accessing memory (if any)
- Our own data blocks and code blocks
- Larger overlap in hit and miss latencies

Mitigating Errors ($0 \rightarrow 1$)

Mitigating Errors ($0 \rightarrow 1$)

- Root privileges are granted for running the program. Needed for setting high affinity and pinning the program to a particular core $\Rightarrow$ *Reduce context switches*

Mitigating Errors (0 → 1)

- Root privileges are granted for running the program. Needed for setting high affinity and pinning the program to a particular core ⇒ *Reduce context switches*
- No other user process is running ⇒ *Reduces noise*

Mitigating Errors ($0 \rightarrow 1$)

- Root privileges are granted for running the program. Needed for setting high affinity and pinning the program to a particular core $\Rightarrow$ *Reduce context switches*
- No other user process is running $\Rightarrow$ *Reduces noise*
- The machine is booted in single user mode

Mitigating Errors (0 → 1)

- Root privileges are granted for running the program. Needed for setting high affinity and pinning the program to a particular core ⇒ *Reduce context switches*
- No other user process is running ⇒ *Reduces noise*
- The machine is booted in single user mode
- Touch all the blocks beforehand ⇒ *Avoid page faults*

Mitigating Errors (0 → 1)

- Root privileges are granted for running the program. Needed for setting high affinity and pinning the program to a particular core ⇒ *Reduce context switches*
- No other user process is running ⇒ *Reduces noise*
- The machine is booted in single user mode
- Touch all the blocks beforehand ⇒ *Avoid page faults*
- Don't use syscalls (for eg., printf) during communication.

Cause of Errors (1 → 0)

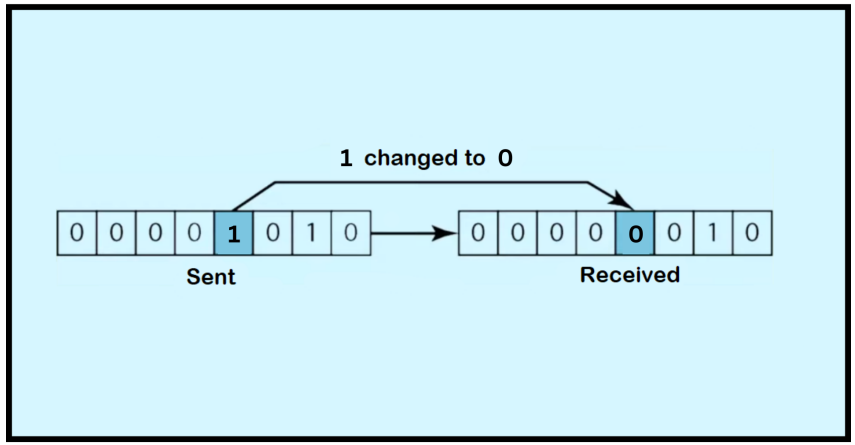


Figure: 1 to 0 errors

Cause of Errors (1 → 0)

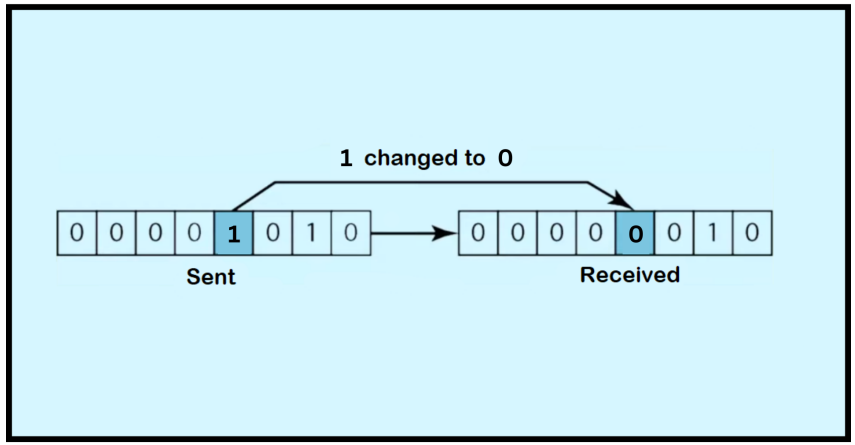


Figure: 1 to 0 errors

- Poor occupancy achieved by receiver

Cause of Errors (1 → 0)

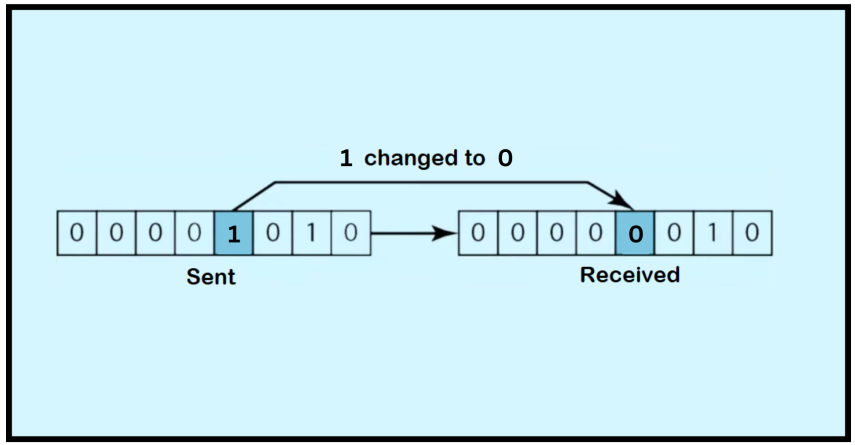


Figure: 1 to 0 errors

- Poor occupancy achieved by receiver
- Poor hit/miss threshold

Cause of Errors (1 → 0)

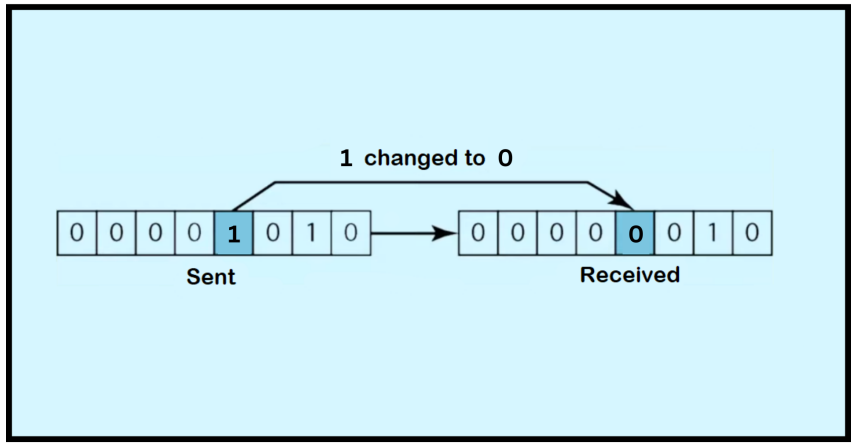


Figure: 1 to 0 errors

- Poor occupancy achieved by receiver
- Poor hit/miss threshold
- Replacement policy

Cause of Errors ($1 \rightarrow 0$)

Cause of Errors ($1 \rightarrow 0$)

- In practical scenarios, replacement policy is not random.

Cause of Errors ($1 \rightarrow 0$)

- In practical scenarios, replacement policy is not random.
- 16 way set $\Rightarrow$ easier to achieve 100% occupancy.

Cause of Errors ($1 \rightarrow 0$)

- In practical scenarios, replacement policy is not random.
- 16 way set $\Rightarrow$ easier to achieve 100% occupancy.
- If the eviction set and DS elements map to the same set in the LLC, any disturbance created by the sender will indeed evict atleast one of the receiver's blocks.

Cause of Errors ($1 \rightarrow 0$)

- In practical scenarios, replacement policy is not random.
- 16 way set $\Rightarrow$ easier to achieve 100% occupancy.
- If the eviction set and DS elements map to the same set in the LLC, any disturbance created by the sender will indeed evict atleast one of the receiver's blocks.
- The overlap in hit and miss latencies should be the main culprit.

Results & Observations

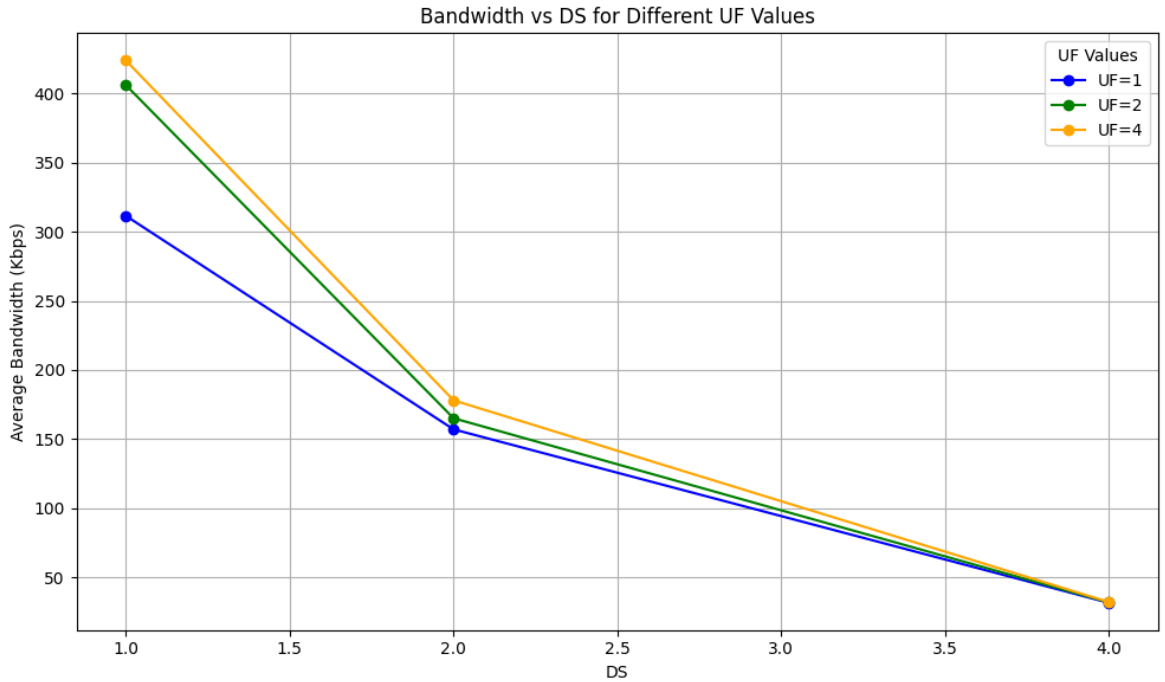


Figure: Bandwidth vs DS

- Bandwidth decreases on increasing DS (significantly)
- Bandwidth increases on increasing unrolling factor (slightly)

Results & Observations

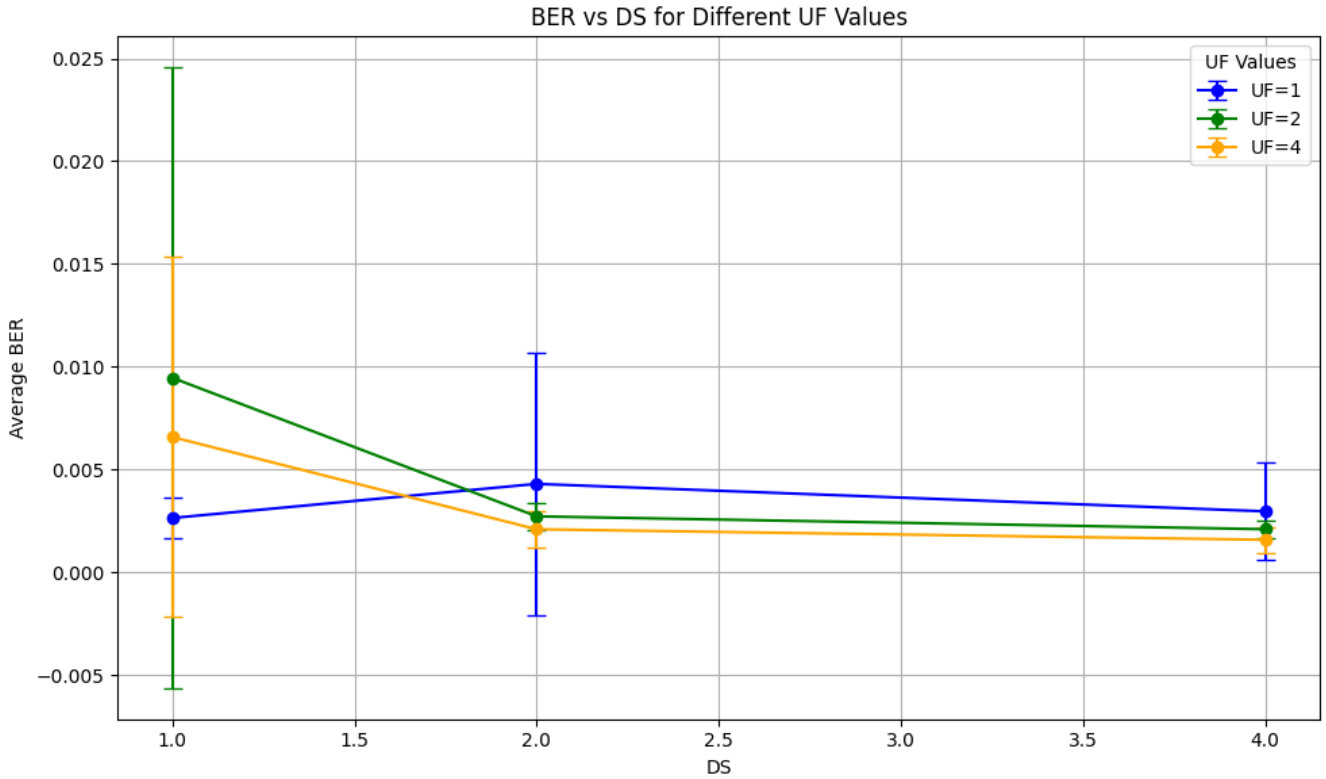


Figure: BER vs DS

- BER decreases on increasing disturbance set size

Results & Observations

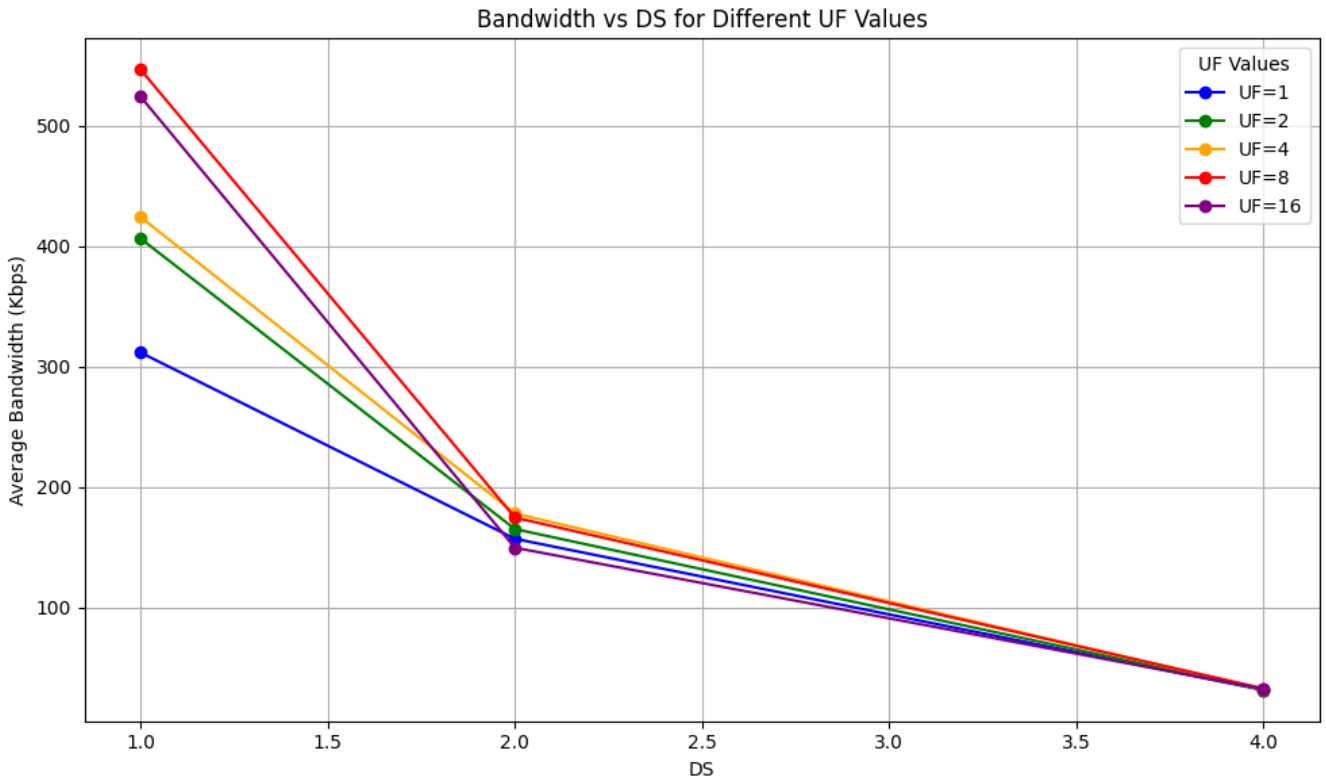


Figure: Bandwidth vs DS

- UF = 8, 16 with DS = 1 have best bandwidths out of all configurations

Results & Observations

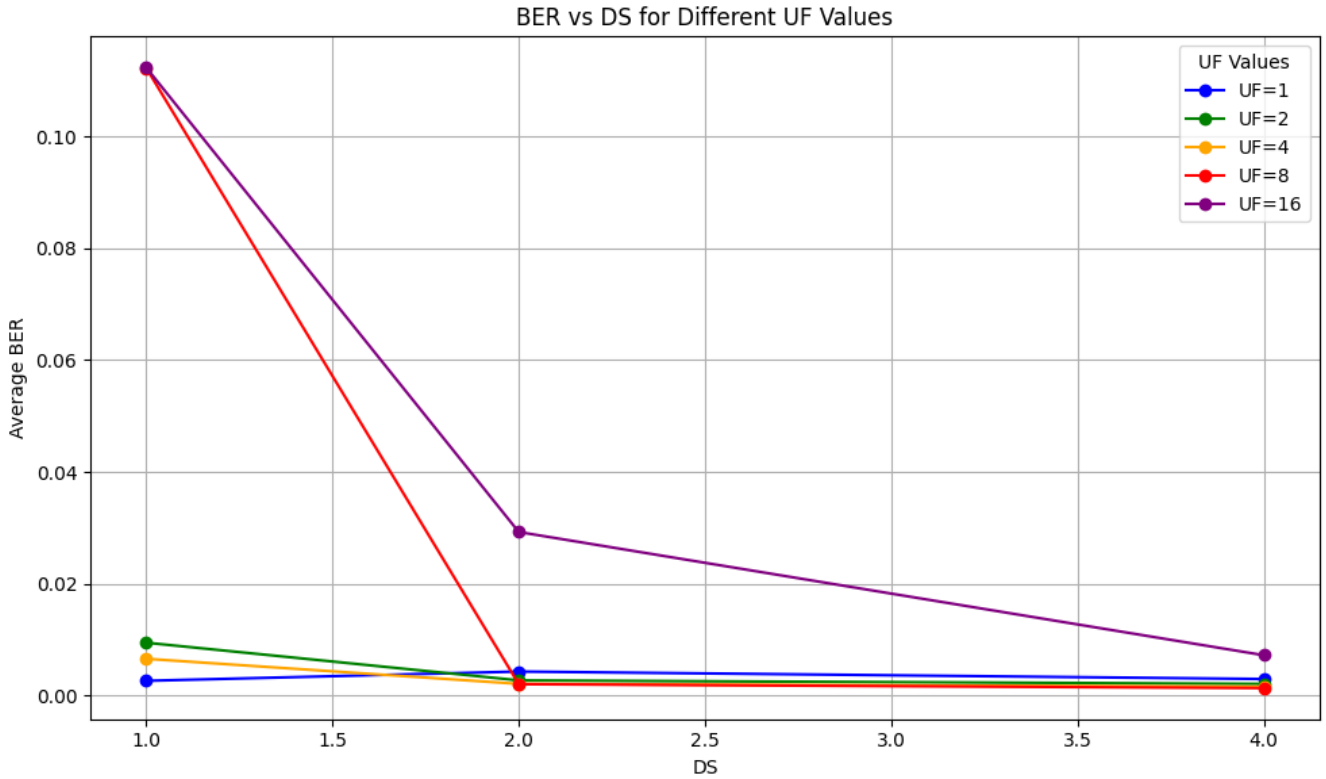


Figure: BER vs DS

- But, the BER is too large!

Future Improvements

- Synchronization of sender and receiver without pipes.
- Ideally there should not be any shared memory between processes.
- Synchronization can be done using the worst case timings for each step.

Thank you for Attending!

Questions and Answers!

Results & Observations

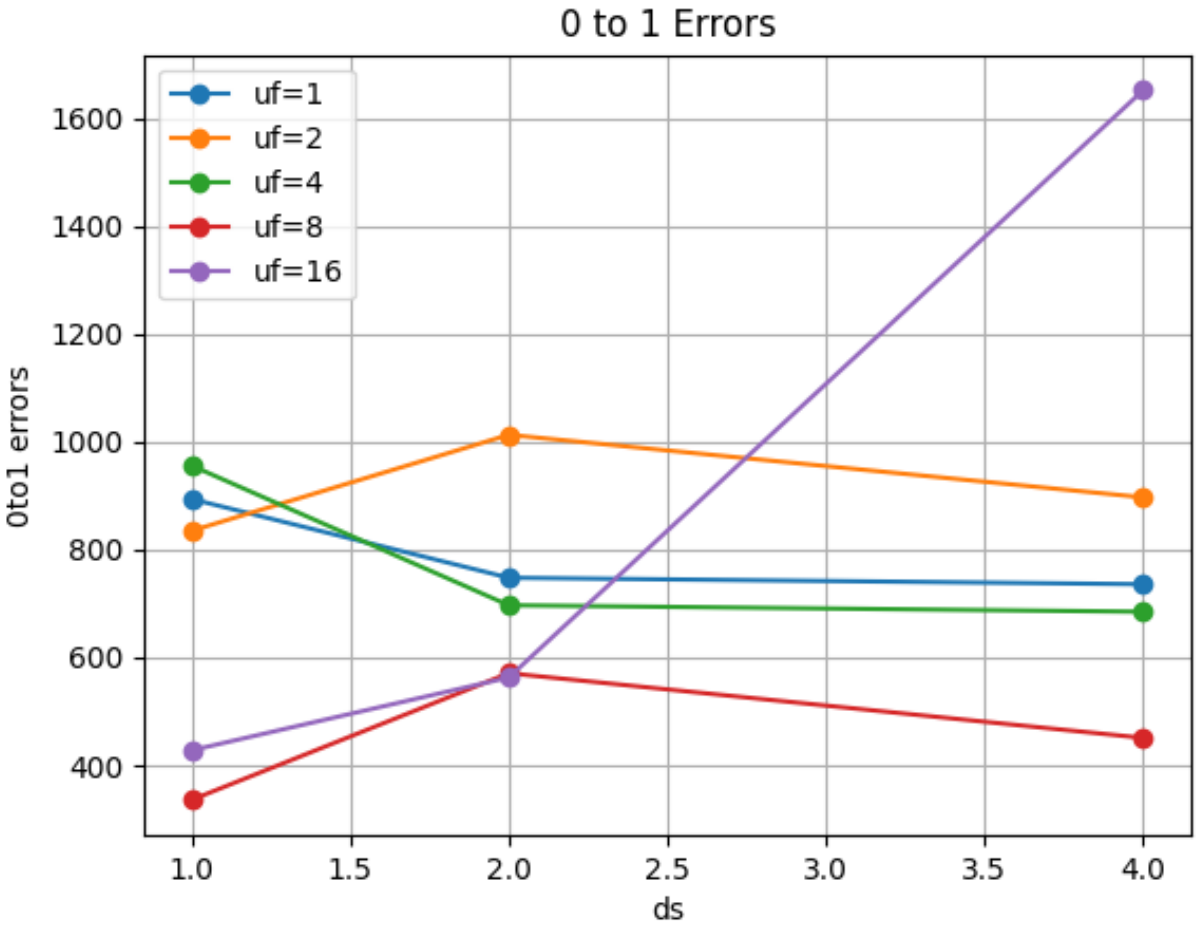


Figure: BER vs DS (0 to 1 errors)

Results & Observations

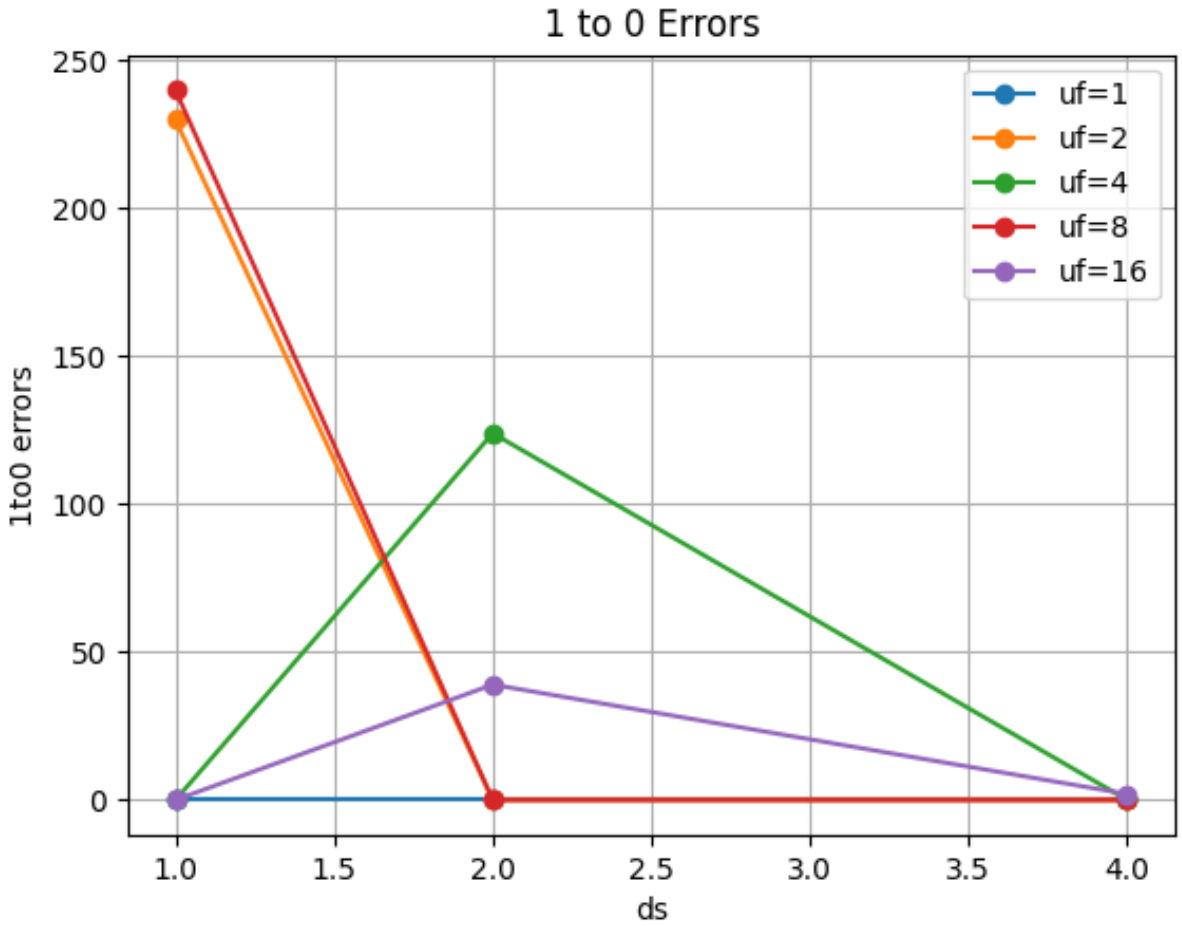


Figure: BER vs DS (1 to 0 errors)

Results & Observations

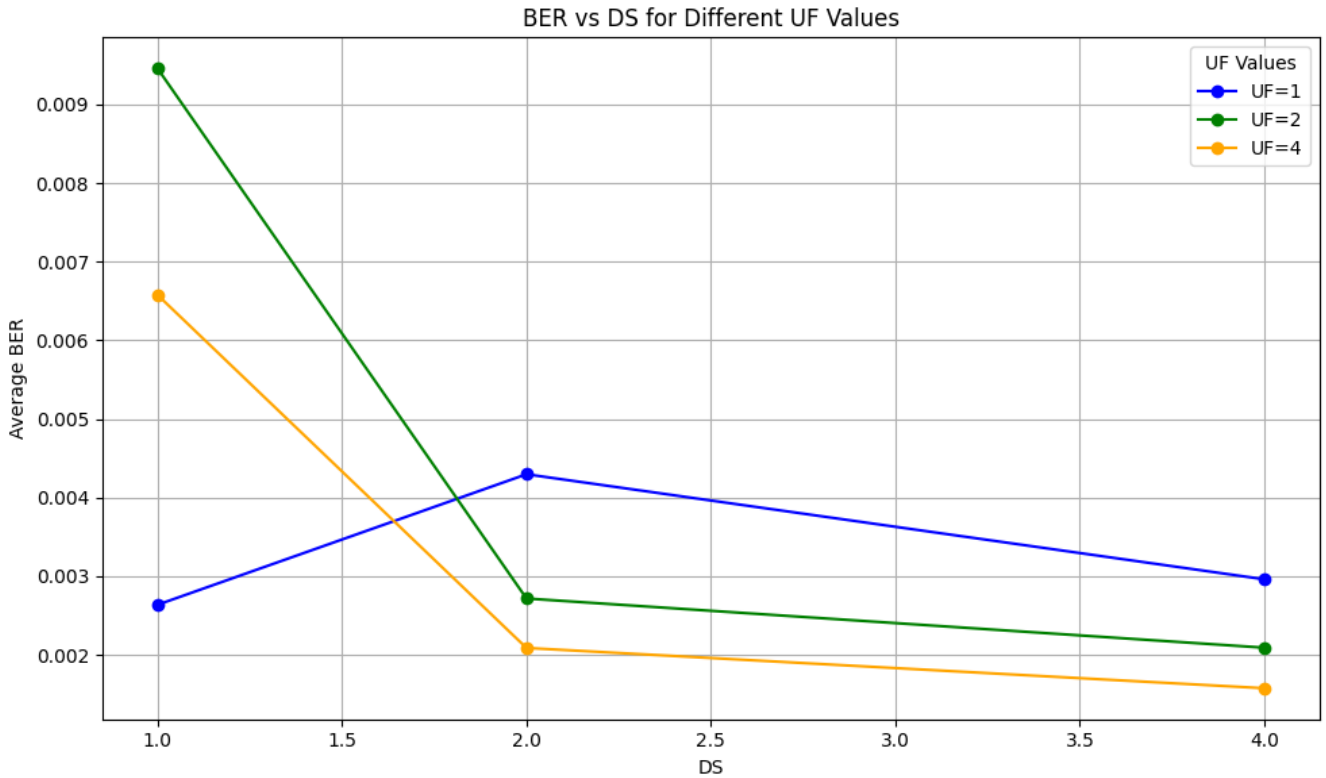


Figure: BER vs DS

Results & Observations

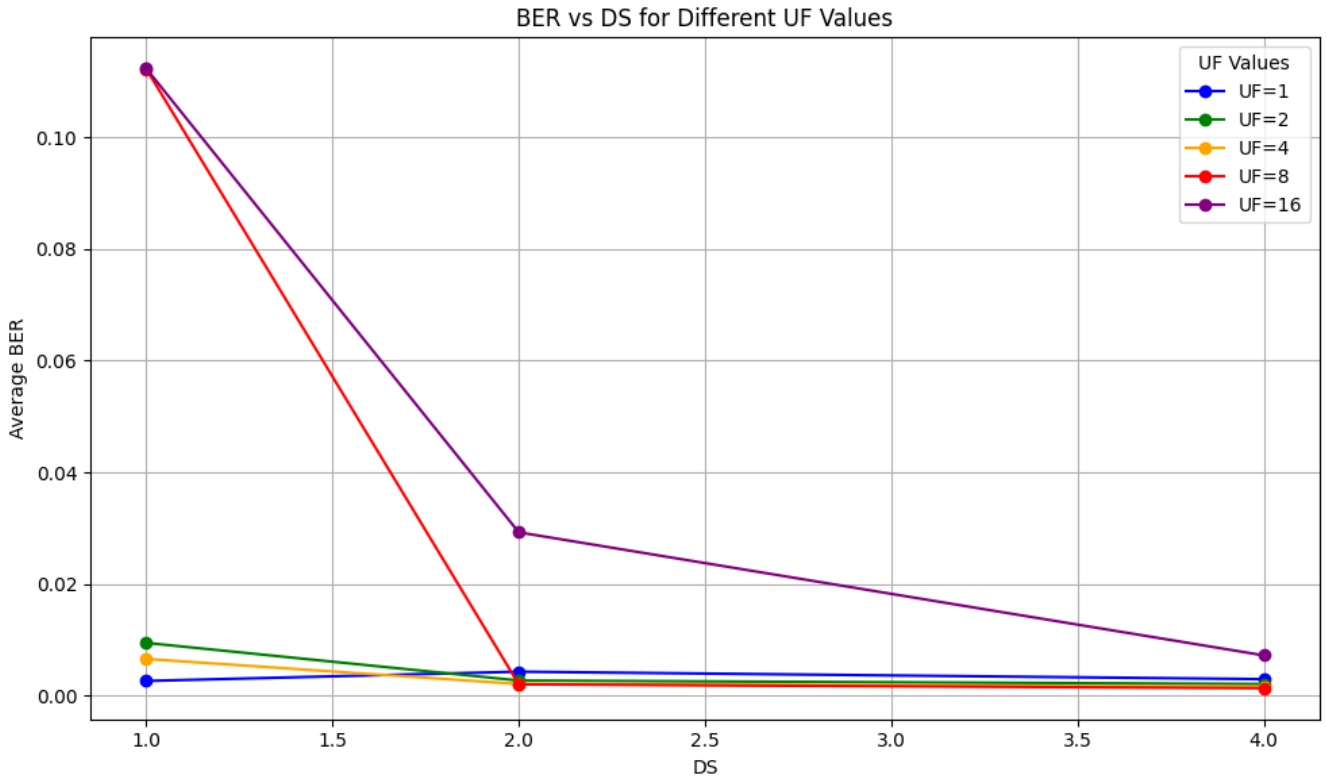


Figure: BER vs DS

Results & Observations

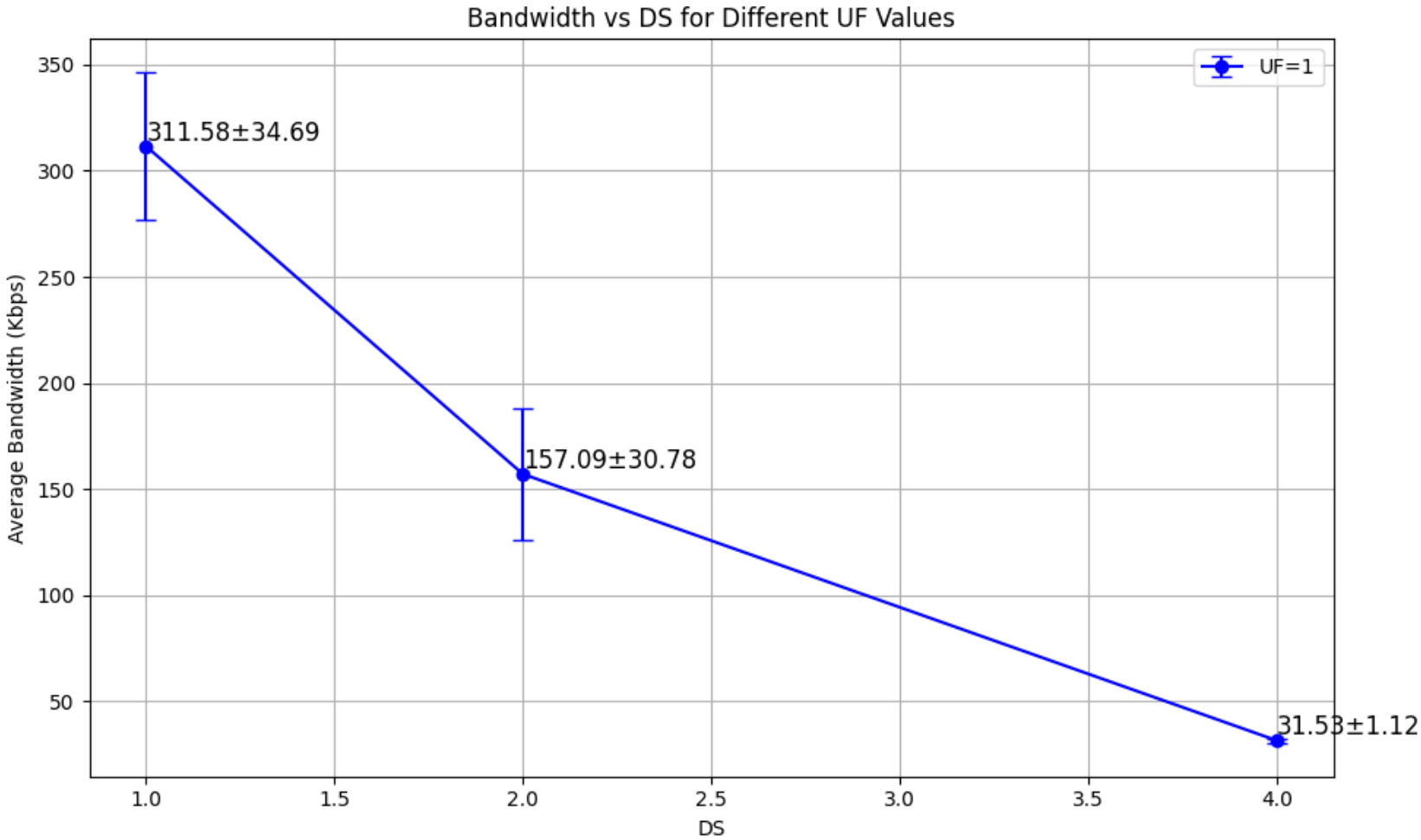


Figure: BW vs DS

Results & Observations

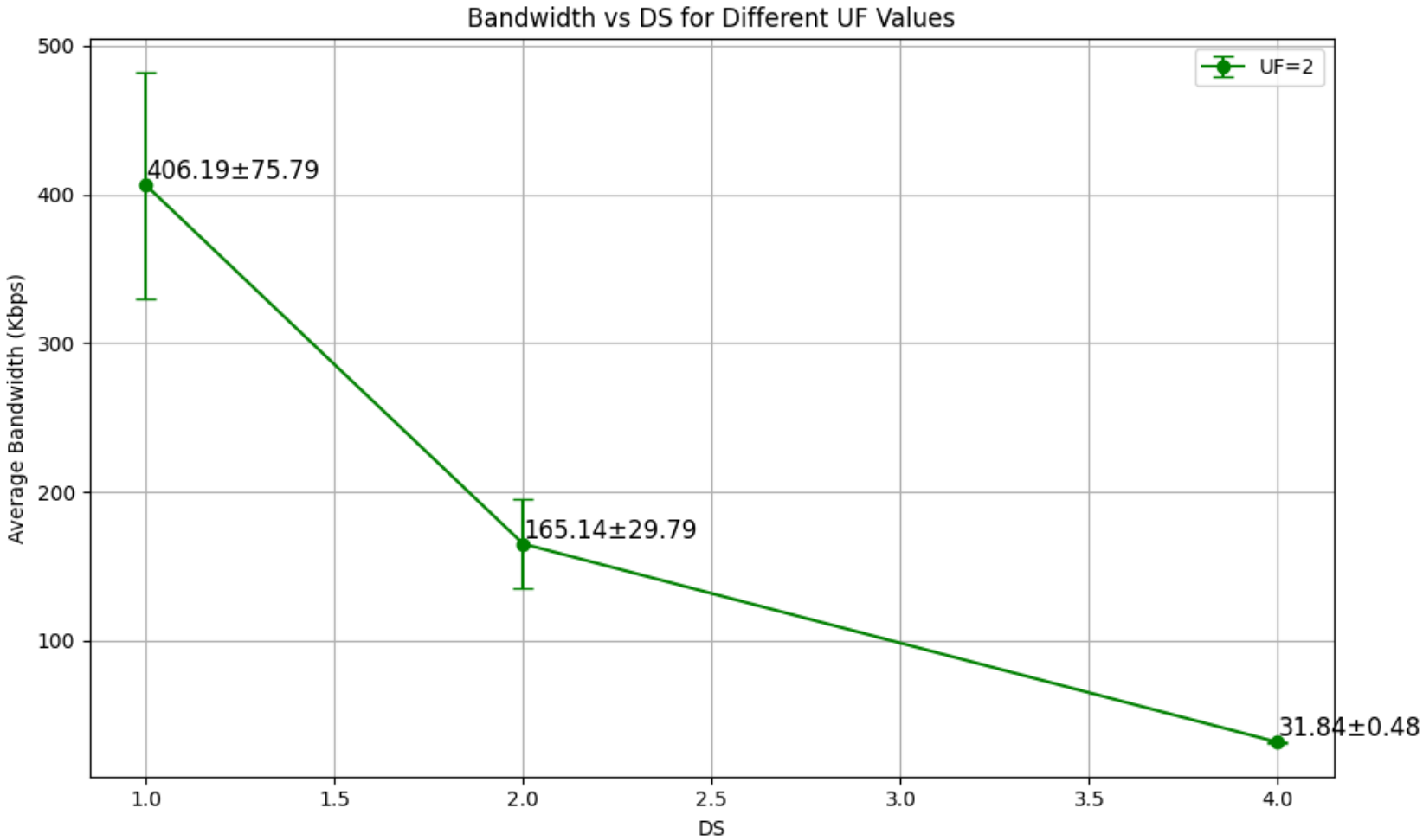


Figure: BW vs DS

Results & Observations

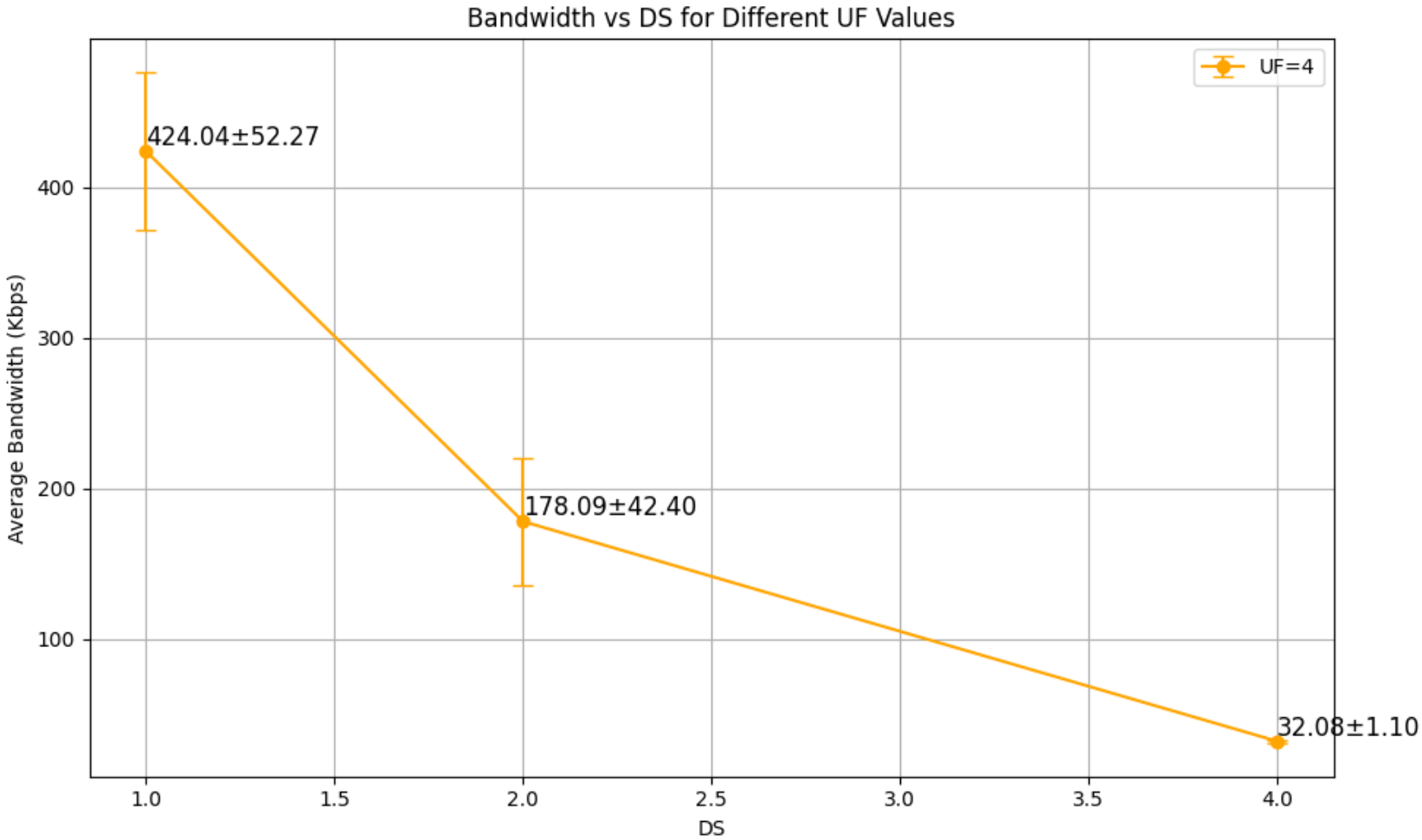


Figure: BW vs DS

Results & Observations

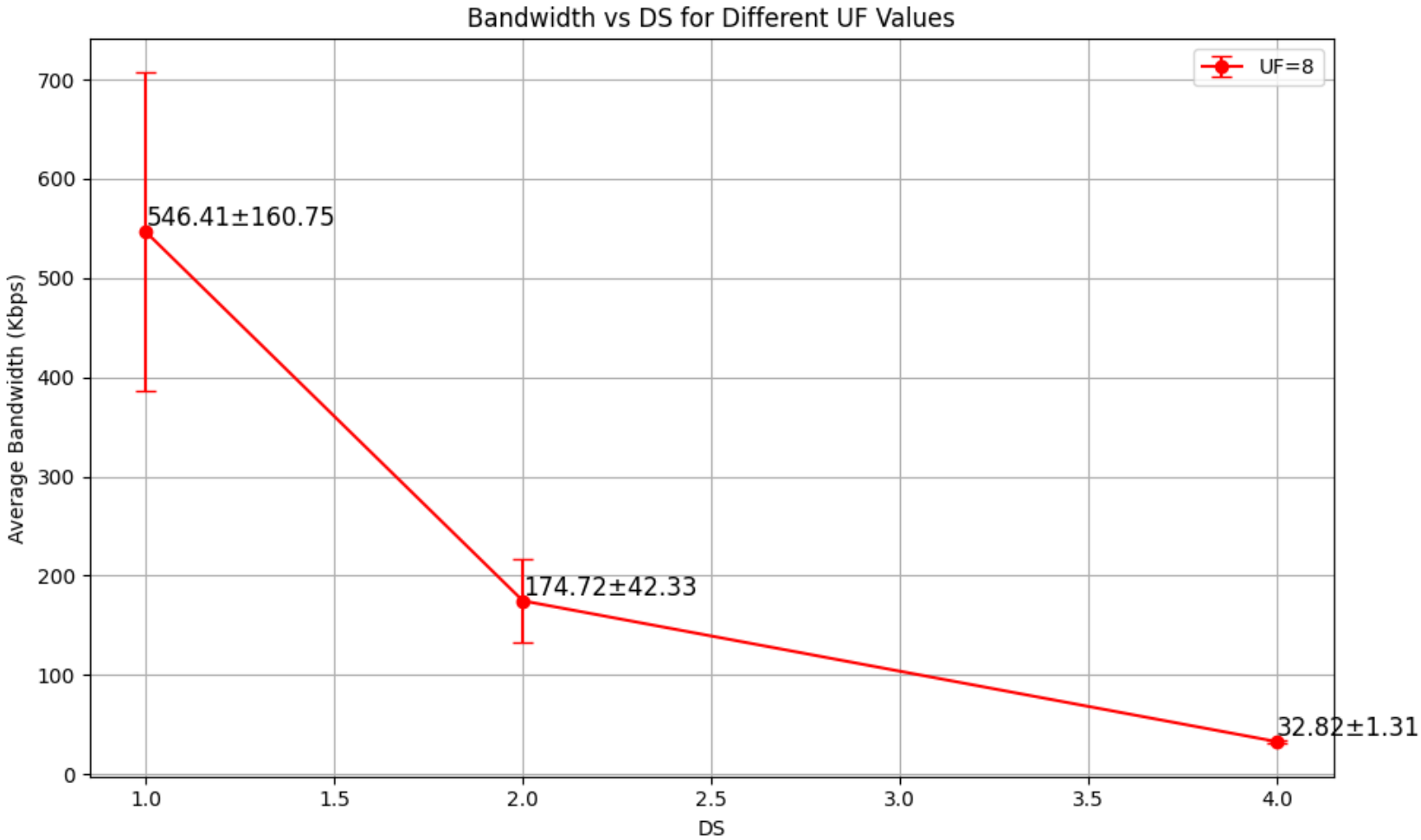


Figure: BW vs DS

Results & Observations

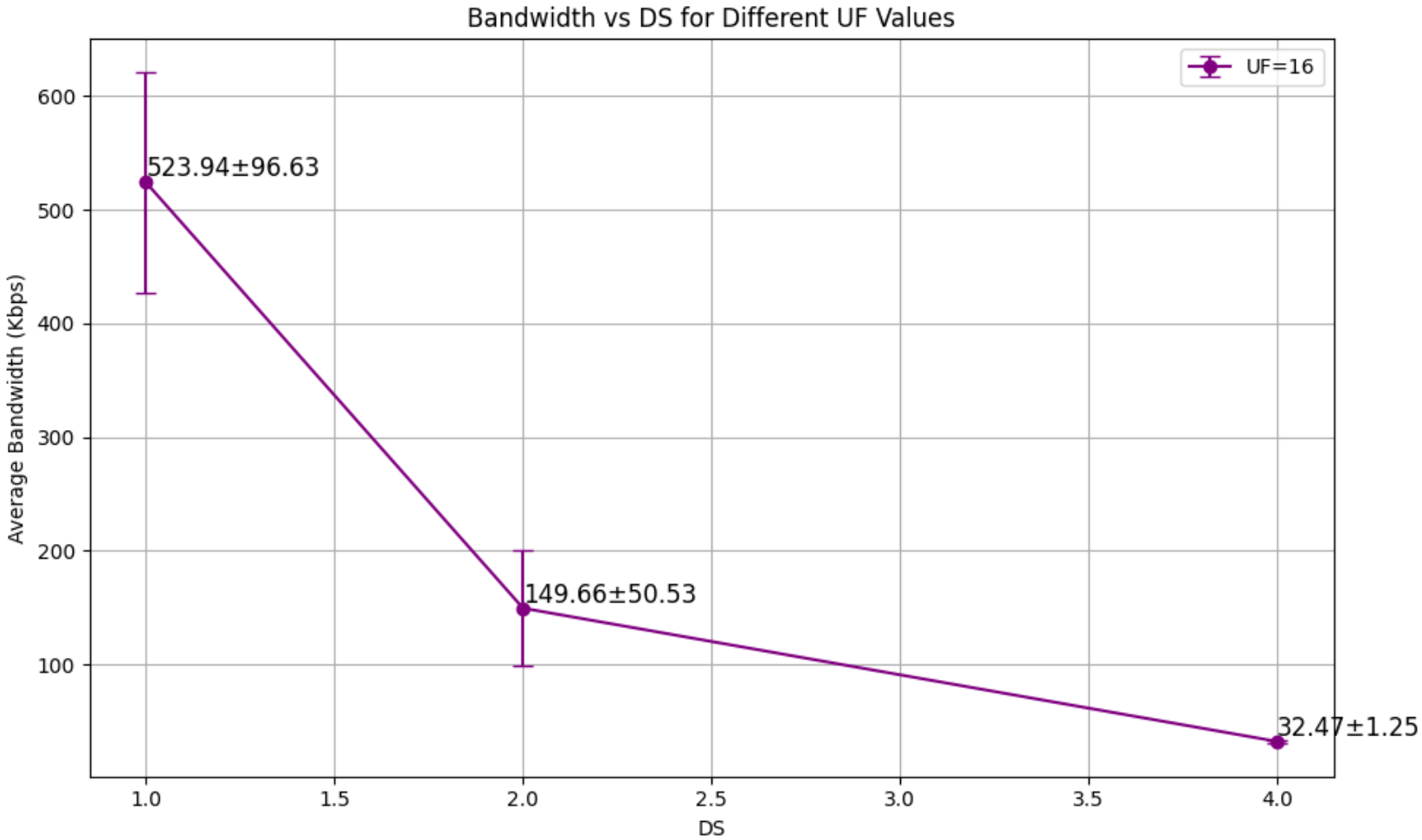


Figure: BW vs DS

Results & Observations

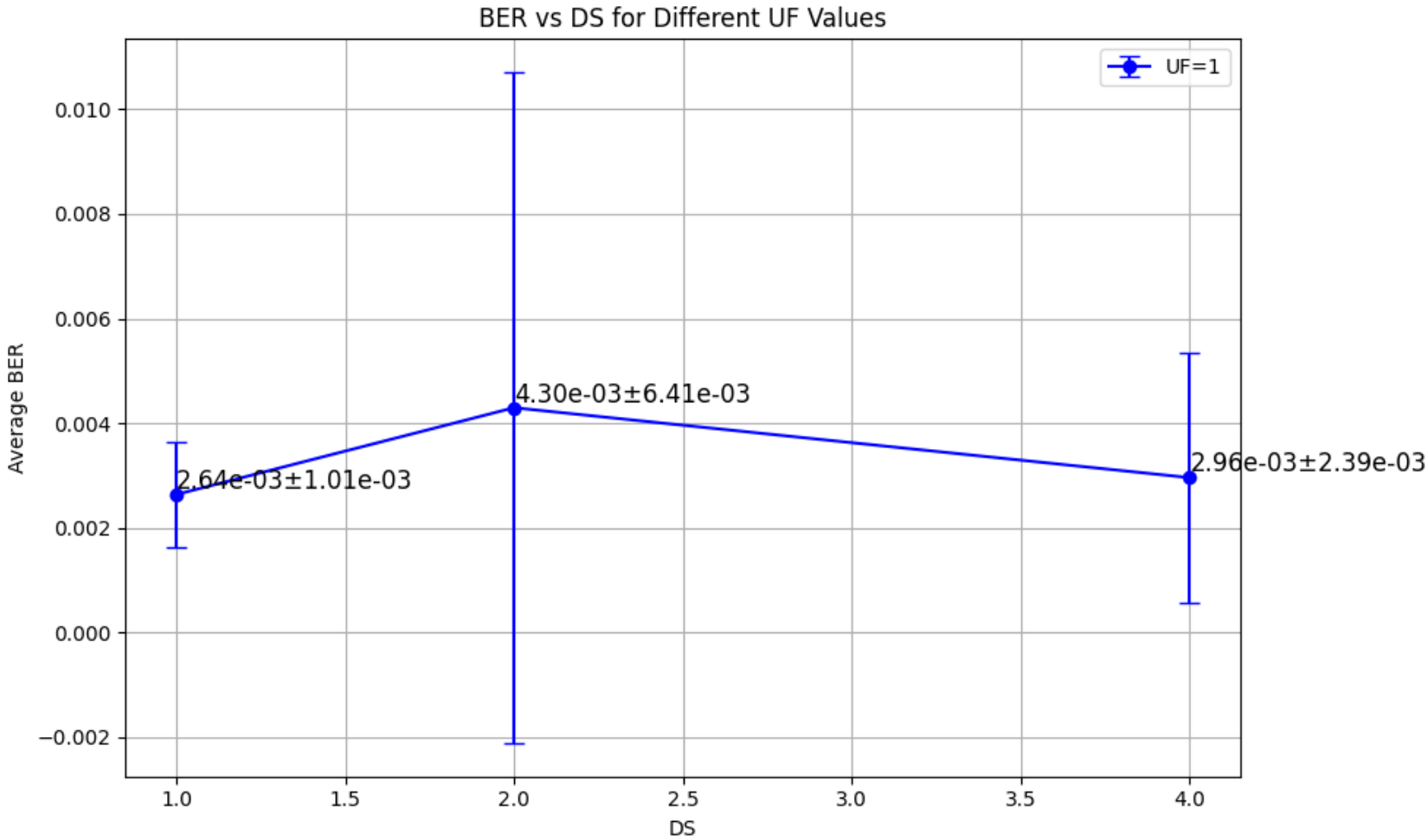


Figure: BER vs DS

Results & Observations

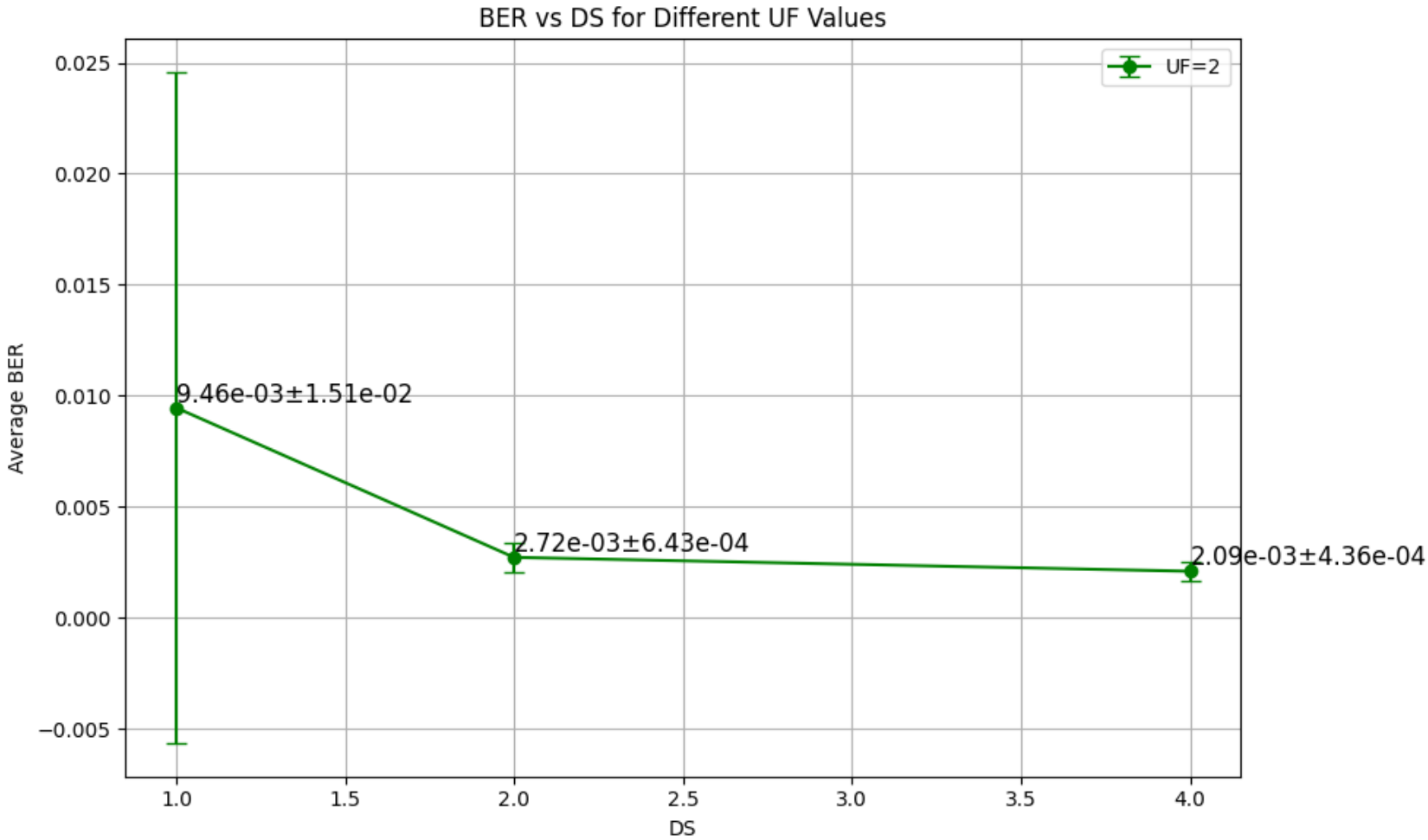


Figure: BER vs DS

Results & Observations

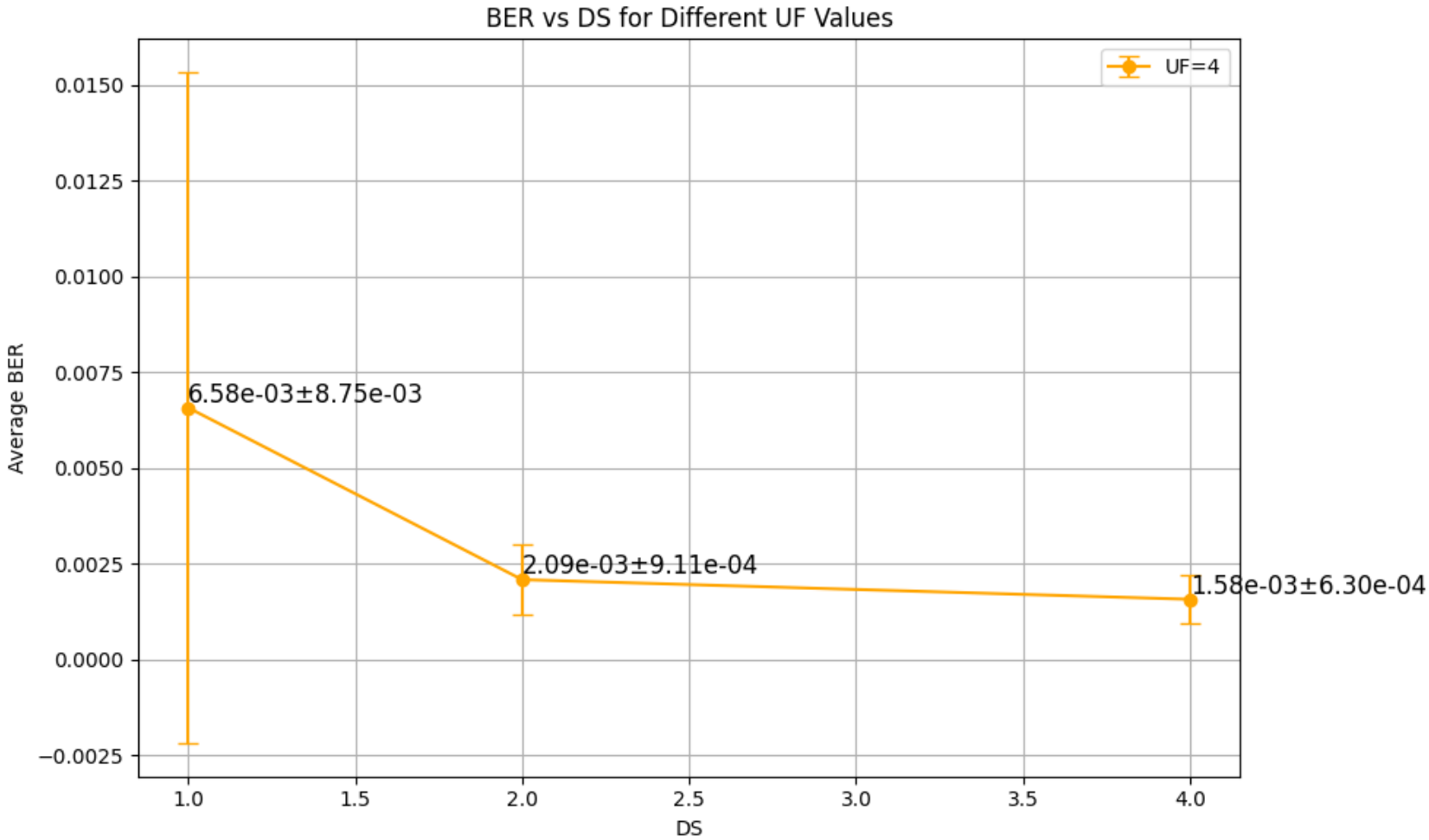


Figure: BER vs DS

Results & Observations

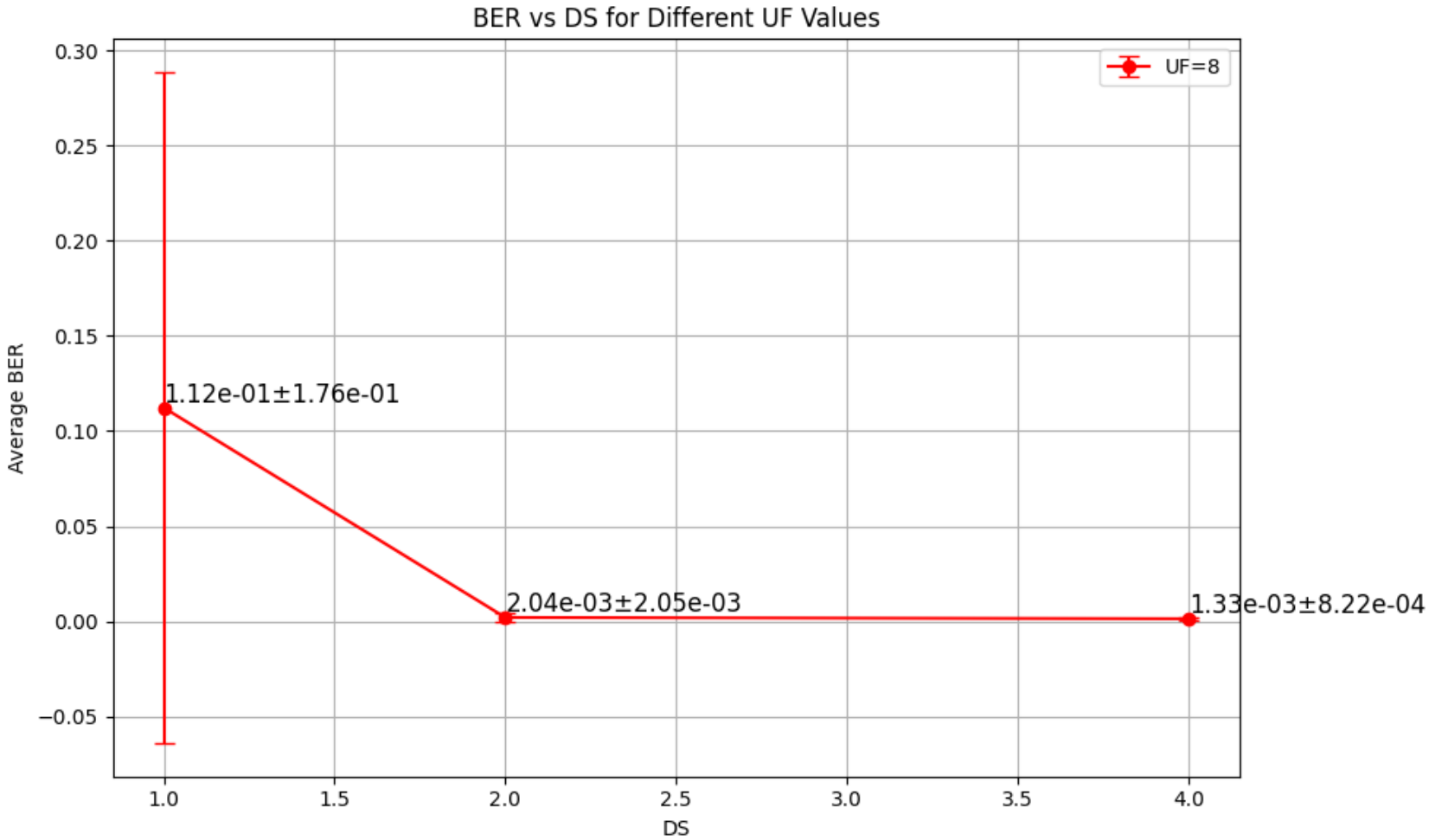


Figure: BER vs DS

Results & Observations

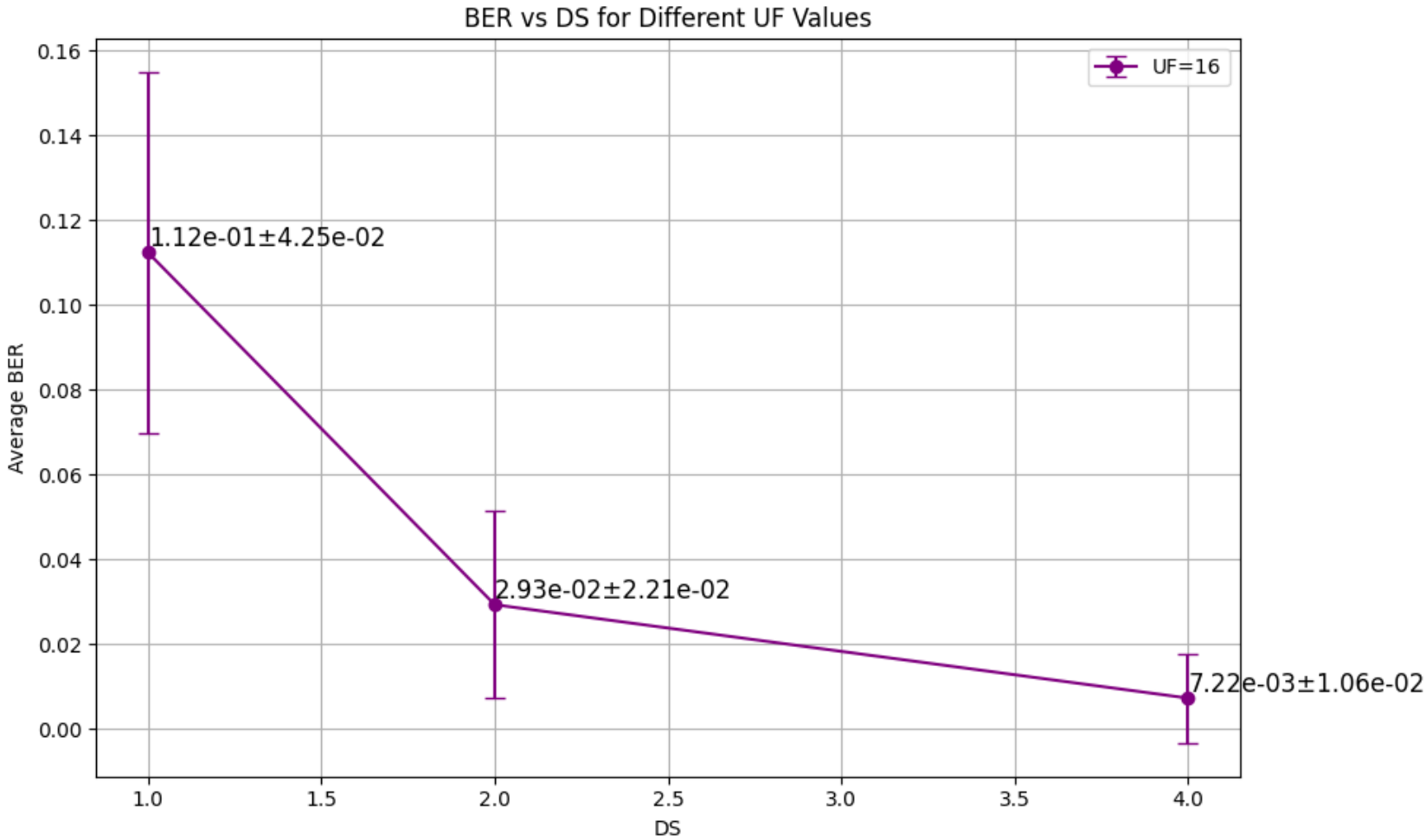


Figure: BER vs DS

Results & Observations

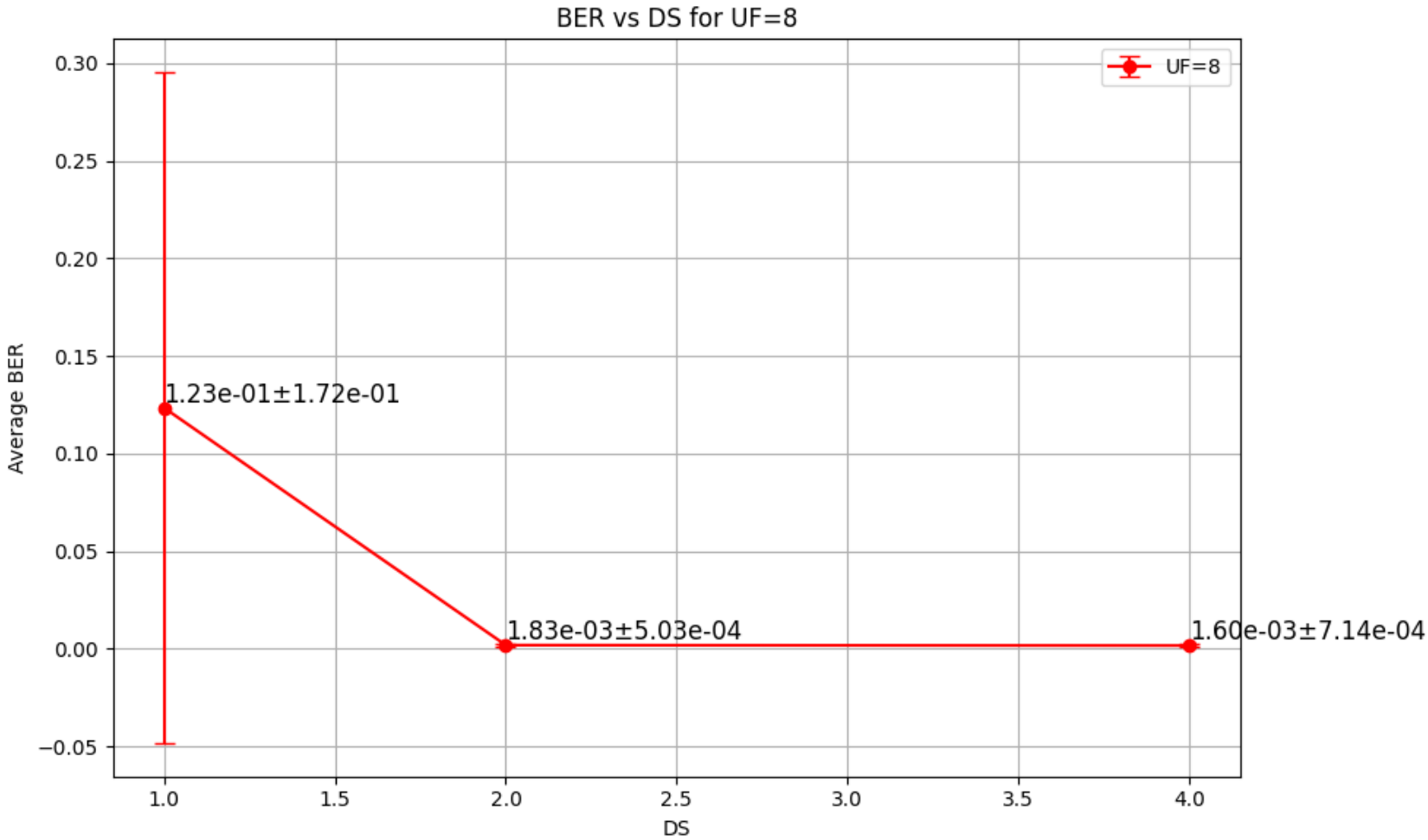


Figure: BER vs DS

Results & Observations

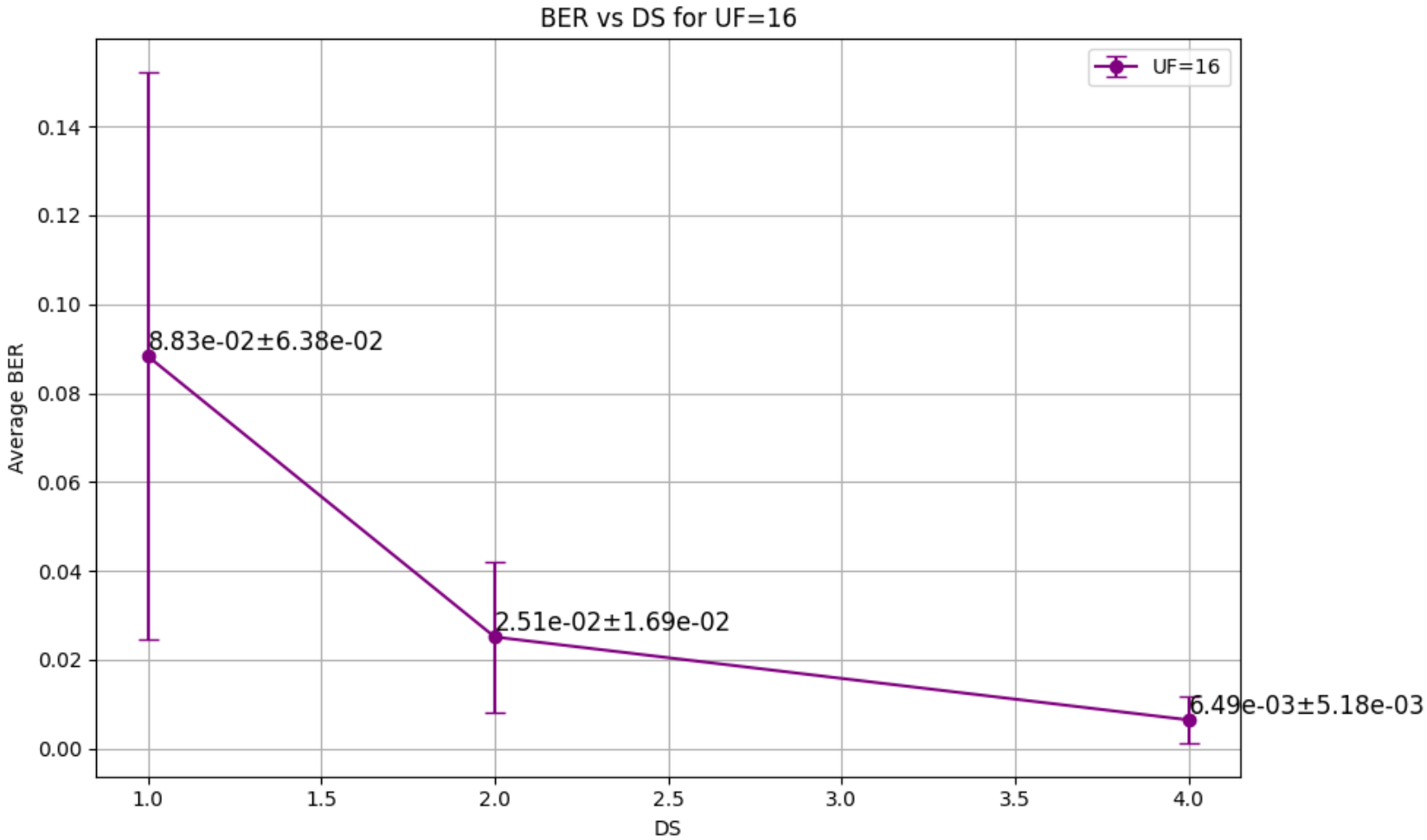


Figure: BER vs DS